Velo

Manual técnico do projeto

E-commerce completo em Java, Spring Boot e React: instalação, telas, painel, pagamentos, integrações, API e banco de dados.

Feito por **Erik L. Pimenta**

Versão 1.15.0 · Outubro de 2026 · Licença MIT

Código aberto

Sumário

1 **Visão geral** 4
- Para que serve 4
- O que a Velo faz 4
- Novidades da versão 1.15.0 5
- Tecnologias 5
- Identidade visual 7

2 **Como rodar** 9
- Primeira execução 9
- Gerar o arquivo .jar 9
- Desenvolvendo a interface 9
- Estrutura de pastas 10

3 **Acesso e sessão** 11
- Contas de demonstração 11
- Regras da sessão 13

4 **Guia da loja** 14
- Vitrine 14
- Produto 15
- Carrinho 15
- Entrega e pagamento 16
- Pagamento por PIX 17
- Pedidos 18
- Minha conta 19
- No celular 20

5 **Painel administrativo** 22
- Perfis e permissões 22
- Visão geral 22
- Paleta de comandos 23
- Pedidos 24
- Produtos e estoque 25
- Exportação CSV 26
- Categorias, cupons e usuários 26

6 **Pedidos e pagamentos** 27
- Ciclo do pedido 27
- Regras de valor e estoque 28
- Pagamento simulado 28
- Stripe 29

Mercado Pago 29

7 **Integrações** 31
O que se integra 31
Tela Integrações 31
Webhooks de saída 32
Conferindo a assinatura 33
CEP e prazo de entrega 34

8 **Arquitetura** 35
Backend por pacote 35
Interface por pasta 36
Decisões principais 36
Erros 37

9 **API** 38

10 **Banco de dados** 41
Pareando com outro banco 41

11 **Configuração, testes e automação** 42
Variáveis de ambiente 42
Testes 42
Integração contínua 43
Commits automáticos 43
Documentação em PDF 43

12 **Solução de problemas** 44

CAPÍTULO 1

Visão geral

A Velo é uma loja virtual completa e de código aberto. Ela cobre o caminho inteiro de uma venda on-line: o cliente encontra o produto, monta o carrinho, paga e acompanha a entrega, enquanto a equipe cuida de pedidos, estoque, cupons, usuários e integrações num painel próprio.

Para que serve

O projeto foi feito para ser usado, estudado e adaptado. Qualquer pessoa pode baixar o código, rodar com um comando, entrar com as contas de demonstração e ver uma loja funcionando, com dados de exemplo, pagamentos simulados e um painel administrativo completo. Para colocar uma loja real no ar basta trocar o banco, as senhas e, se quiser cobrar de verdade, ligar a Stripe ou o Mercado Pago.

O que a Velo faz

Para quem compra

- Vitrine com faixa de benefícios, busca (atalho `/`), categorias em abas com contagem, ordenação, paginação, favoritos e vistos recentemente.
- Página de produto com estoque em tempo real, preço anterior riscado, simulador de parcelas e cálculo de frete e prazo pelo CEP.
- Carrinho em gaveta lateral e em página completa, com barra de frete grátis e cupom de desconto.
- Checkout com indicador de etapas, endereço preenchido pelo CEP, prévia do cartão com detecção de bandeira, cartão em até 10 vezes, PIX ou boleto.
- Acompanhamento do pedido com linha do tempo, QR Code do PIX, rastreio, comprovante para impressão, nova tentativa de pagamento e cancelamento.
- Conta com dados pessoais, endereços e troca de senha, e tema claro ou escuro.

Para quem vende

- Visão geral com faturamento, ticket médio, variação contra o período anterior em cada indicador, mix de formas de pagamento, vendas por dia, mais vendidos, estoque baixo e pedidos recentes.
- Pedidos com busca, filtros, mudança de situação, código de rastreio, cancelamento, estorno e exportação CSV.
- Produtos com cadastro completo, movimentação de estoque (entrada, saída e contagem), histórico e exportação CSV.
- Categorias, cupons (percentual ou valor fixo, com mínimo, limite de usos e validade) e usuários com três perfis.
- Paleta de comandos (`Ctrl+K` ou `⌘K`) para ir a qualquer tela ou abrir um pedido pelo número.
- Tela de integrações com a situação dos provedores de pagamento, URLs de webhook, webhooks de saída e consulta de CEP.

Por baixo

- Estoque reservado com trava no banco no momento da compra e devolvido sozinho ao cancelar ou estornar.
- Pedidos não pagos em 60 minutos são cancelados automaticamente.
- Sessão com limite de 20 minutos parado e 30 minutos no total, conferida pelo servidor a cada chamada.
- Pagamento simulado, Stripe Checkout ou Mercado Pago Checkout Pro, todos com webhook assinado.
- Webhooks de saída assinados com HMAC-SHA256 para avisar outros sistemas a cada mudança do pedido.
- Um único modelo de banco, versionado com Flyway, que roda em seis bancos diferentes.

Novidades da versão 1.15.0 NOVO

A 1.15.0 traz uma nova identidade visual e uma camada de integrações. O banco de dados não muda: nenhuma migração nova é necessária para atualizar.

Área	O que mudou
Interface	Mesma logo e mesma paleta (preto, branco, cinzas e verde-pinho), com Instrument Serif nos títulos, tela de entrada em duas colunas, cartões de produto maiores e tema escuro.
Vitrine e produto	Faixa de benefícios, abas com contagem, favoritos, vistos recentemente, atalho `/`, simulador de parcelas e frete com prazo pelo CEP.
Checkout e pedido	Indicador de etapas, endereço pelo CEP (ViaCEP), prévia do cartão com validação em tempo real, QR Code do PIX e comprovante para impressão.
Painel	Variação contra o período anterior, mix de pagamentos, exportação CSV, paleta de comandos e a nova tela Integrações.
Integrações	Mercado Pago Checkout Pro, webhooks de saída assinados, consulta de CEP e cotação de frete.
API	Sete rotas novas: CEP, cotação de frete, exportações, integrações, evento de teste e webhook do Mercado Pago.

Tecnologias

Camada	Tecnologia	Papel
Linguagem	Java 21	Toda a regra de negócio e a API
API	Spring Boot 4.0.8 (Web MVC, Security, Validation, Data JPA)	Rotas REST, segurança, validação e acesso a dados
Banco	Hibernate 7 + Flyway	Mapeamento das tabelas e criação do esquema
JSON	Jackson 3	Conversão de dados e leitura dos webhooks da Stripe e do Mercado Pago
Interface	React 19 + TypeScript 5.9	Telas da loja e do painel
Build da interface	Vite 7	Servidor de desenvolvimento e pacote final

Camada	Tecnologia	Papel
Animações	Motion 12	Gaveta do carrinho, modais e avisos, sempre em resposta a uma ação
Fontes	Instrument Serif e Public Sans	Títulos de vitrine e produto; interface e números tabulares
Scripts	Python 3	Gerador do esquema do banco, das ilustrações e destes PDFs
Testes	JUnit 6 + Spring Boot Test	Sessão, compra, permissões e regras
Bancos	H2, PostgreSQL, MySQL, MariaDB, SQL Server, Oracle	Escolha de quem instala

Identidade visual

A interface é sóbria, feita para quem usa a loja e o painel todos os dias. A logo e a paleta continuam as mesmas: preto e branco, fundo cinza bem claro, superfícies brancas com bordas finas e um único tom de destaque, o verde-pinho, que aparece nos gráficos, na seleção, nas etapas concluídas e no foco do teclado.



`#F6F7F7`
Fundo das páginas

`#FFFFFF`
Cartões, tabelas e formulários

`#E2E5E6`
Bordas e divisórias

`#5F666B`
Textos de apoio

`#15181B`
Texto, botões e logo

`#1F5C4C`
Destaque, foco e gráficos

Instrument Serif

Títulos de vitrine e de produto. Peso único, usado em tamanhos grandes.

Public Sans

R$ 1.249,90 · 10x R$ 124,99

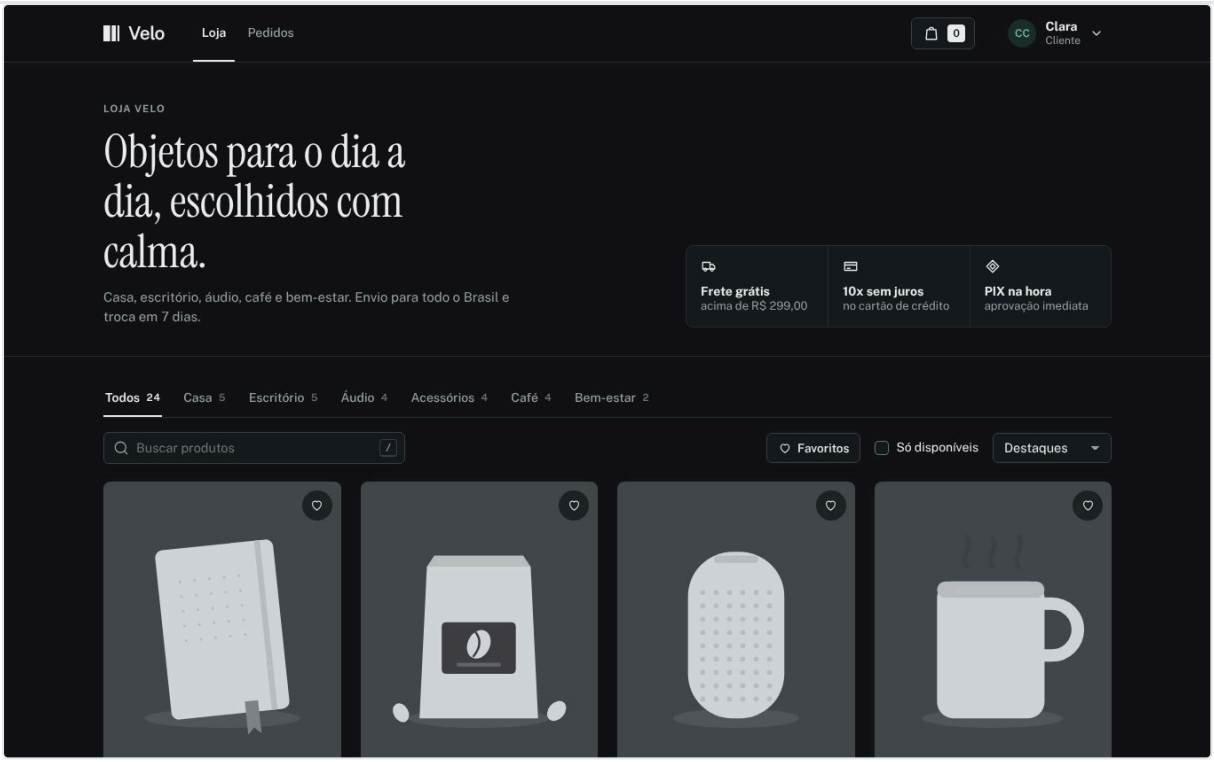
Interface, textos e números, sempre tabulares em preços e totais.

As cores de estado aparecem só em etiquetas, avisos e variações de valor: sucesso `#1F6B45`, aviso `#8A5A12` e erro `#A03A32`. Os cantos têm de 4 a 8 px, os botões são retangulares e as situações dos pedidos aparecem em etiquetas discretas. As animações acontecem só em resposta a uma ação (abrir o carrinho, um modal ou um aviso); trocar de página não tem efeito de entrada nem cascata.

As ilustrações dos produtos são em preto sobre cinza claro e são geradas por `scripts/imagens/gerar_imagens.py`. A marca, em preto e branco, são três barras de larguras decrescentes; ela aparece desenhada em vetor no cabeçalho, no painel, na tela de entrada e no ícone da aba, e as versões em imagem ficam em `docs/images/logo-velo-500.png` e `docs/images/logo-transparente.png` (sem fundo).

Tema escuro

O tema segue a preferência do sistema operacional e pode ser trocado no menu do usuário ou na paleta de comandos. No escuro, a paleta se inverte: fundo grafite, texto e logo em branco, e o verde-pinho é clareado para manter o contraste.



Vitrine no tema escuro

CAPÍTULO 2

Como rodar

Só é preciso ter o Java 21 instalado. O Maven baixa o Node, compila a interface e embute tudo num único servidor.

Primeira execução

Windows

1. Instale o Java 21 (por exemplo, o Temurin 21 da Adoptium) e confirme com `java -version`.
2. Abra a pasta do projeto e dê dois cliques em `iniciar.cmd`.
3. Espere aparecer `Started VeloApplication` na janela. A primeira vez leva alguns minutos para baixar as dependências.
4. Abra http://localhost:8080 no navegador e entre com uma das contas de demonstração.

Linux ou macOS

```
./iniciar.sh
```

Sem configurar nada, a Velo usa um banco H2 em arquivo, em `backend/data`, já preenchido com os dados de demonstração. Para começar do zero, pare a aplicação e apague essa pasta.

Gerar o arquivo .jar

```
cd backend
./mvnw -B package
java -jar target/velo.jar
```

No Windows, use `mvnw.cmd` no lugar de `./mvnw`. O .jar já contém a interface e pode ser copiado para qualquer servidor com Java 21.

Desenvolvendo a interface

Para mexer nas telas com recarga instantânea, rode a API sem compilar a interface e use o servidor do Vite:

TERMINAL 1

```
cd backend
./mvnw -B spring-boot:run -DskipUi
```

TERMINAL 2

```
cd frontend
npm install
npm run dev
```

Abra http://localhost:5173. As chamadas para `/api` são encaminhadas para a porta 8080.

Estrutura de pastas

Pasta	Conteúdo
`backend/`	API Spring Boot, migrações do banco, configuração e testes
`backend/src/main/java/dev/velo/`	Código Java dividido por assunto (veja o capítulo de arquitetura)
`backend/src/main/resources/db/migration/`	Um `V1__schema.sql` para cada banco suportado
`frontend/`	Interface React + TypeScript
`frontend/public/products/`	Ilustrações dos 24 produtos de demonstração
`scripts/schema/`	Gerador do esquema e da documentação do banco
`scripts/imagens/`	Gerador das ilustrações dos produtos
`scripts/docs/`	Gerador deste manual e do PDF do banco de dados
`docs/`	Guia do banco, PDFs da documentação e imagens da marca e das telas
`docker/` e `docker-compose.yml`	Bancos prontos para teste com Docker
`.github/workflows/`	Integração contínua e commits automáticos

CAPÍTULO 3

Acesso e sessão

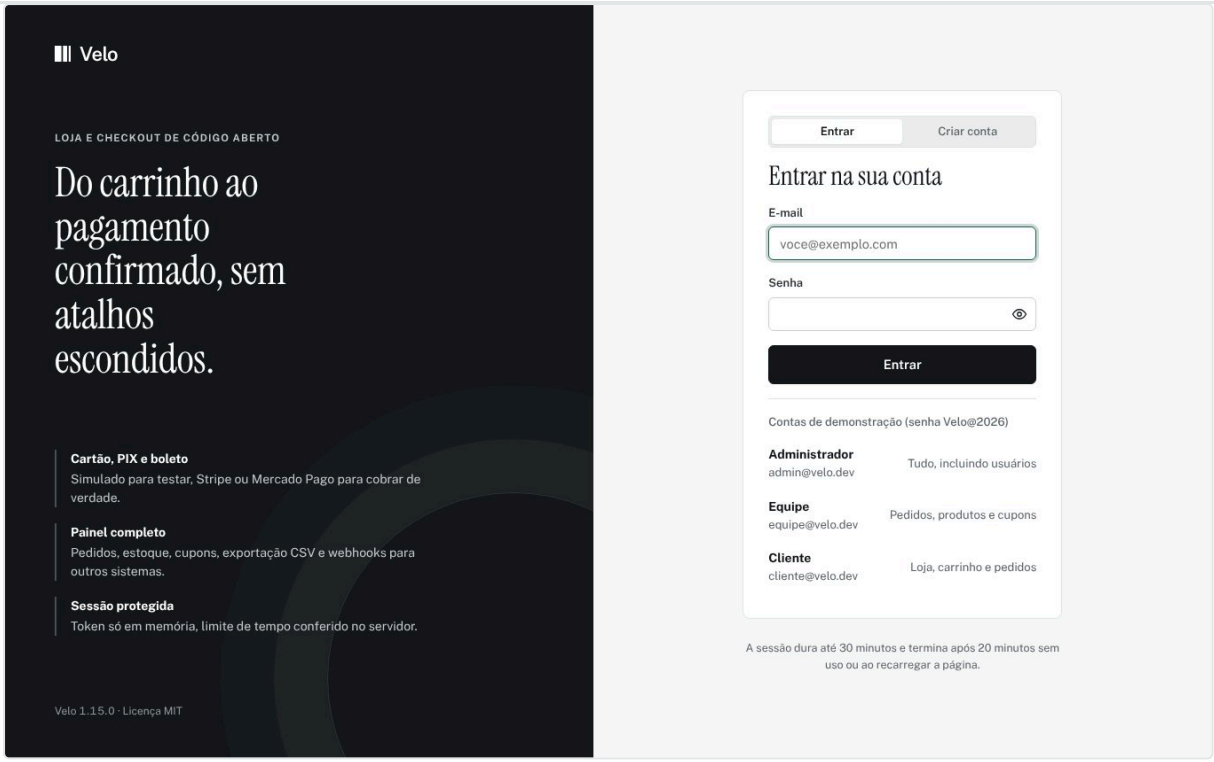
Toda tela e toda rota da API exigem login. A sessão é curta de propósito e é conferida tanto pelo navegador quanto pelo servidor.

Contas de demonstração

Todas usam a senha `Velo@2026`. Elas estão no código de propósito, para qualquer pessoa conseguir entrar logo depois de baixar o projeto. A tela de entrada tem duas colunas: à esquerda, um painel preto com a logo em branco e um resumo do que a Velo faz; à direita, o cartão branco com as abas **Entrar** e **Criar conta** e o título **Entrar na sua conta**. As contas de demonstração ficam em lista logo abaixo do formulário: basta clicar numa delas para preencher o e-mail e a senha. Abaixo do cartão, um aviso lembra quanto tempo dura a sessão.

Perfil	E-mail	O que pode fazer
Administrador	`admin@velo.dev`	Tudo, incluindo usuários e a tela Integrações
Equipe	`equipe@velo.dev`	Painel, pedidos, produtos, categorias, cupons e exportações
Cliente	`cliente@velo.dev`	Loja, carrinho, checkout, pedidos e conta

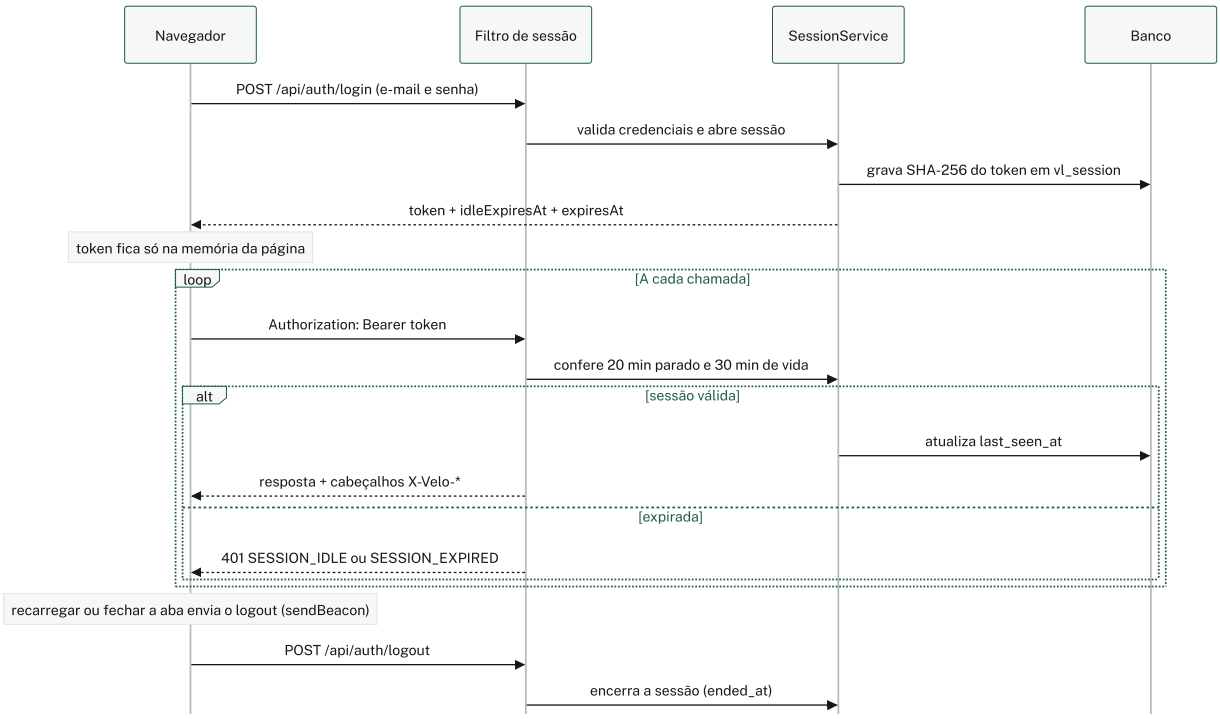
Os dados de demonstração trazem 24 produtos em 6 categorias, 9 clientes fictícios, 64 pedidos de exemplo e os cupons `BEMVINDO10` (10%), `VELO20` (20% acima de R$ 150) e `INVERNO15` (inativo). Para uma loja real, troque essas senhas ou suba com `VELO_SEED=false` num banco vazio.



Tela de entrada com as contas de demonstração

Regras da sessão

Regra	Como funciona
Rotas protegidas	Sem login, qualquer endereço leva à tela de entrada. O painel exige perfil Equipe ou Administrador; as telas Usuários e Integrações só abrem para Administrador.
Recarregou, saiu	O token fica só na memória da página. Recarregar, fechar a aba ou abrir outra aba exige entrar de novo, e a sessão é encerrada no servidor.
20 minutos parado	Sem cliques, teclas ou rolagem por 20 minutos, a sessão termina. No último minuto aparece um aviso com a opção de continuar.
30 minutos no total	Mesmo em uso, a sessão dura no máximo 30 minutos. Um aviso aparece no último minuto.
Conferência no servidor	Os dois limites são verificados pela API a cada chamada. Enquanto a pessoa usa a tela, o navegador avisa o servidor uma vez por minuto.
Bloqueio de senha	Cinco tentativas erradas bloqueiam o login daquele e-mail por 5 minutos.
Senhas e tokens	Senhas ficam só como hash BCrypt; sessões guardam apenas o SHA-256 do token.



Como uma sessão é aberta, conferida e encerrada

Os limites ficam em `application.yml` (`velo.session.idle-minutes` e `velo.session.max-minutes`).

CAPÍTULO 4

Guia da loja

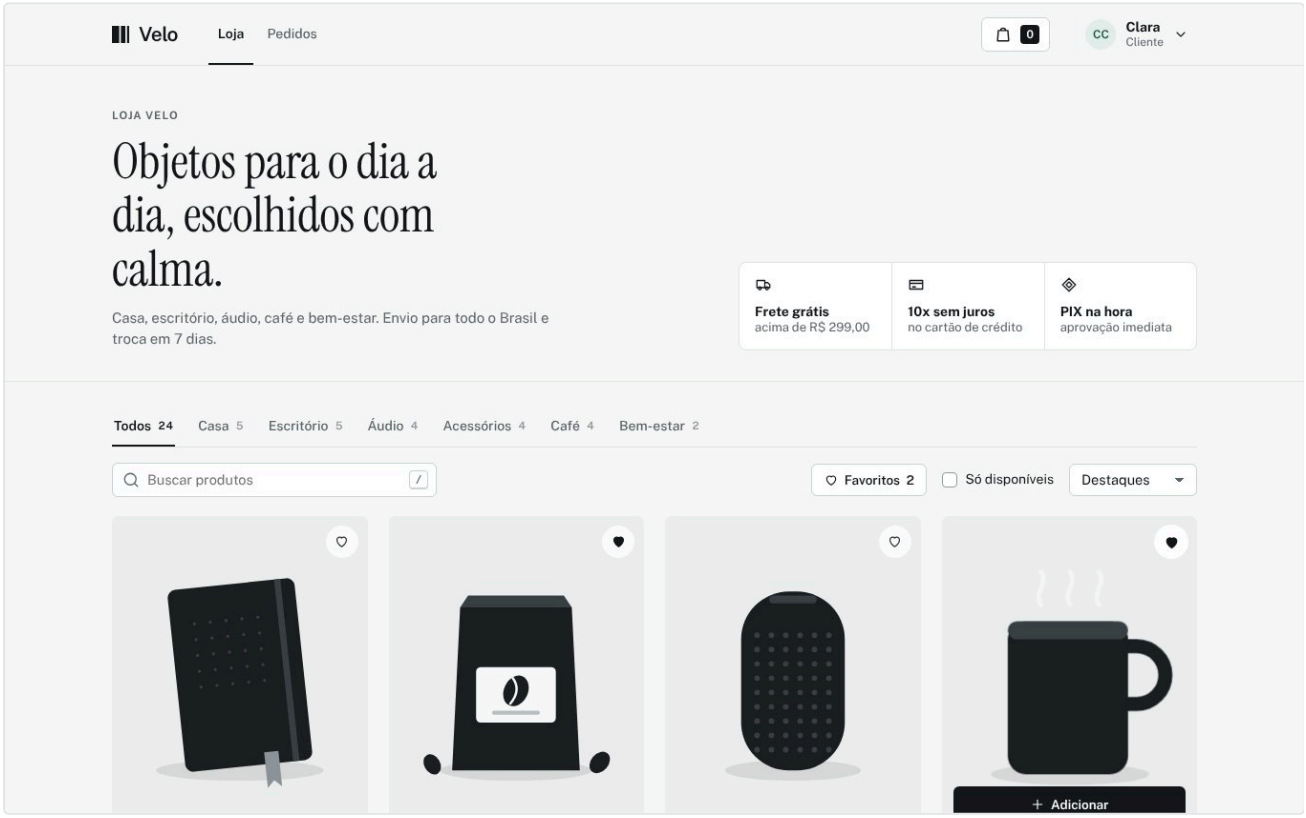
O caminho de quem compra, tela a tela.

Vitrine

O cabeçalho da loja fica fixo no topo, com a logo, os links **Loja** e **Pedidos** (o ativo fica sublinhado), o botão do carrinho com o contador de itens e o menu do usuário. A vitrine abre com um título em Instrument Serif (ou o nome da categoria escolhida) e uma faixa de benefícios: **frete grátis acima de R$ 299**, **até 10x sem juros** e **PIX com aprovação na hora**.

Os produtos aparecem em grade, com cartões verticais, preço e parcela. As categorias ficam em abas com a contagem de itens de cada uma; a busca filtra enquanto se digita, e à direita dá para ver só os favoritos, esconder os esgotados (**Só disponíveis**) e ordenar por destaque, novidade, preço ou nome. Ao passar o mouse num produto, o botão **Adicionar** coloca o item direto no carrinho.

- **Favoritos.** O coração no card marca o produto; o filtro **Favoritos** mostra só os marcados. A lista fica guardada no navegador, separada por usuário.
- **Vistos recentemente.** Uma faixa no fim da vitrine traz os últimos produtos abertos.
- **Atalho de busca.** Em qualquer tela da loja, a tecla `/` leva o cursor para a busca.
- **Promoções.** O preço anterior aparece riscado e o selo de desconto fica em preto, no canto do cartão.

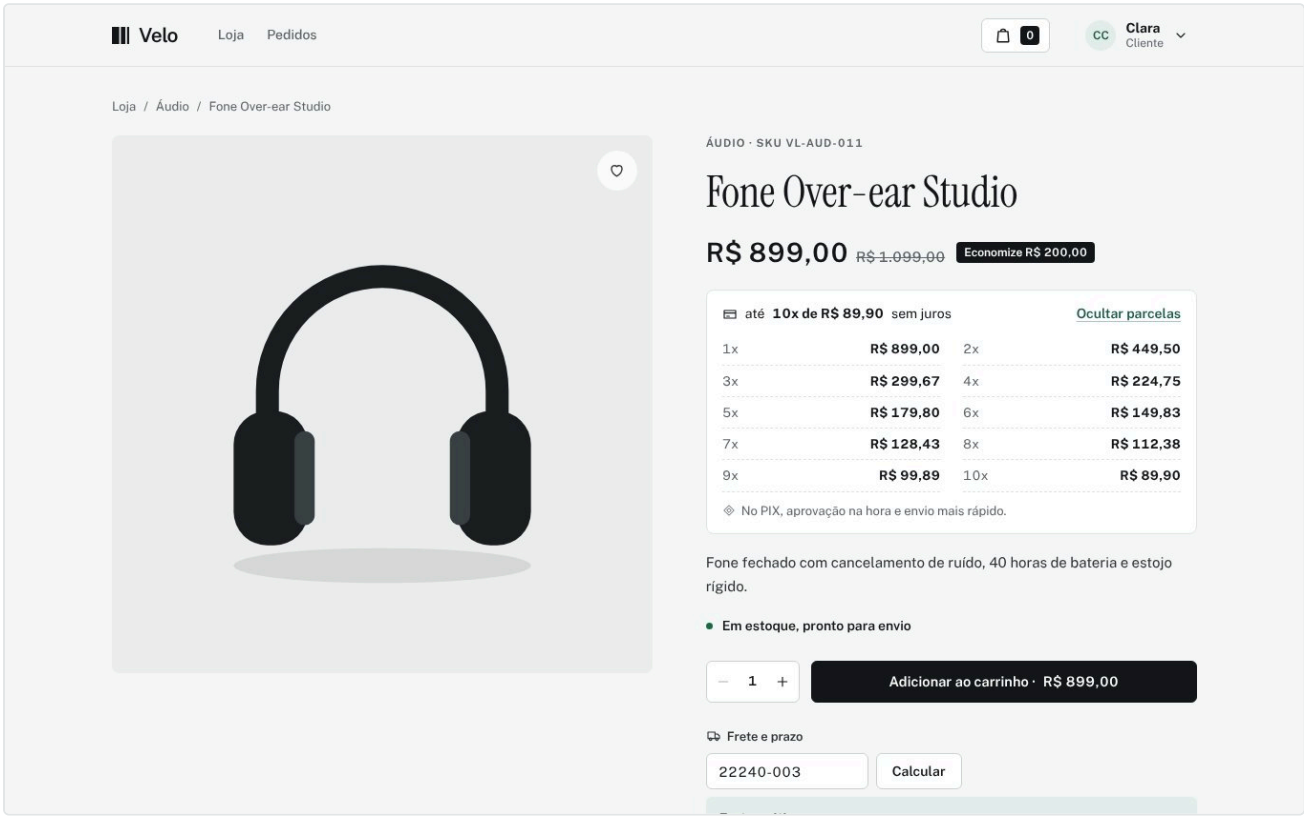


Vitrine com faixa de benefícios, categorias em abas, busca e favoritos

Produto

Mostra a ilustração, o nome em Instrument Serif, a descrição, o preço (com o anterior riscado quando há desconto), o estoque e o seletor de quantidade. Quando restam poucas unidades, a página avisa. O coração adiciona aos favoritos.

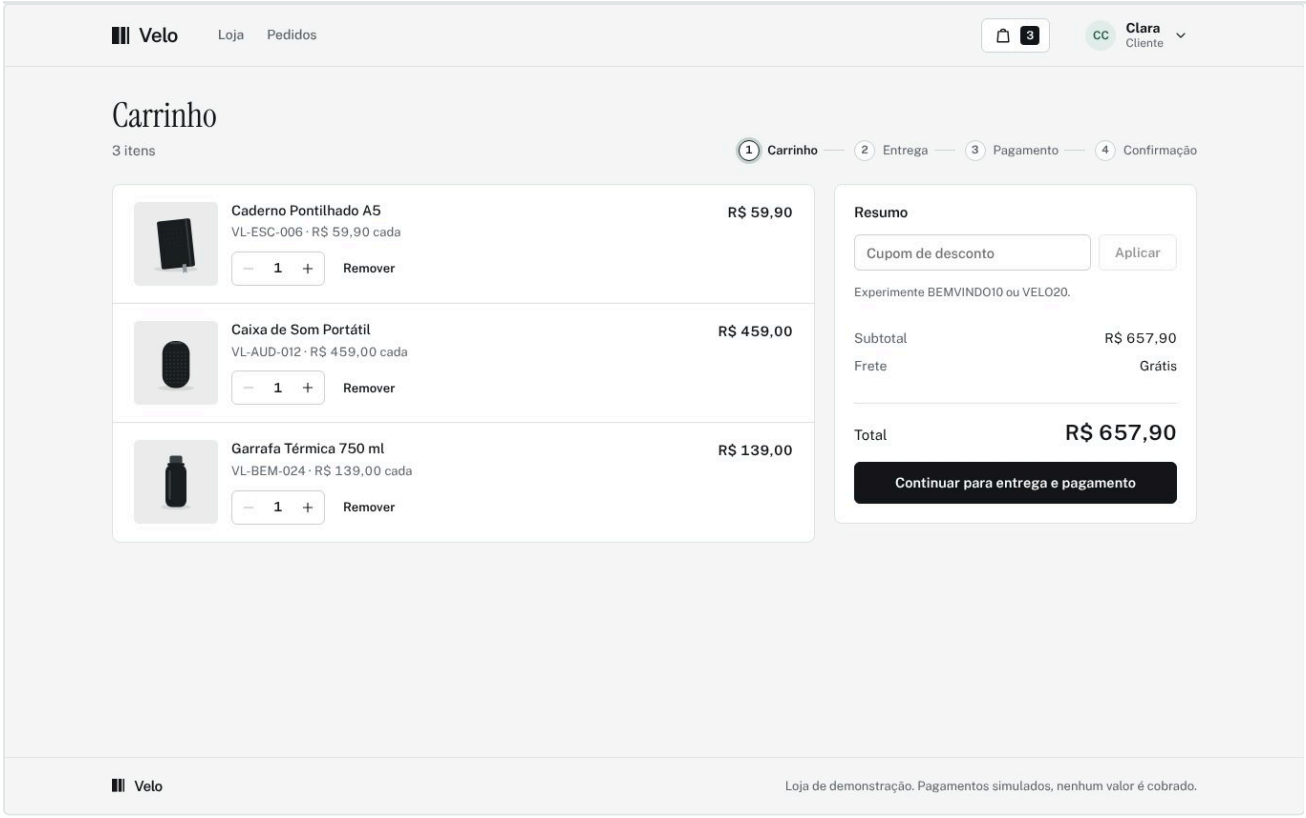
- **Simulador de parcelas.** Mostra o valor de cada parcela de 1x a 10x sem juros.
- **Frete e prazo pelo CEP.** Ao digitar o CEP, a página mostra o valor do frete (R$ 19,90 ou grátis acima de R$ 299) e o prazo estimado para a região.



Página do produto com simulador de parcelas e cálculo de frete

Carrinho

O carrinho abre numa gaveta lateral a partir de qualquer tela, com a barra de frete grátis, e também tem uma página completa com o campo de cupom e o resumo. O frete custa R$ 19,90 e é grátis a partir de R$ 299.



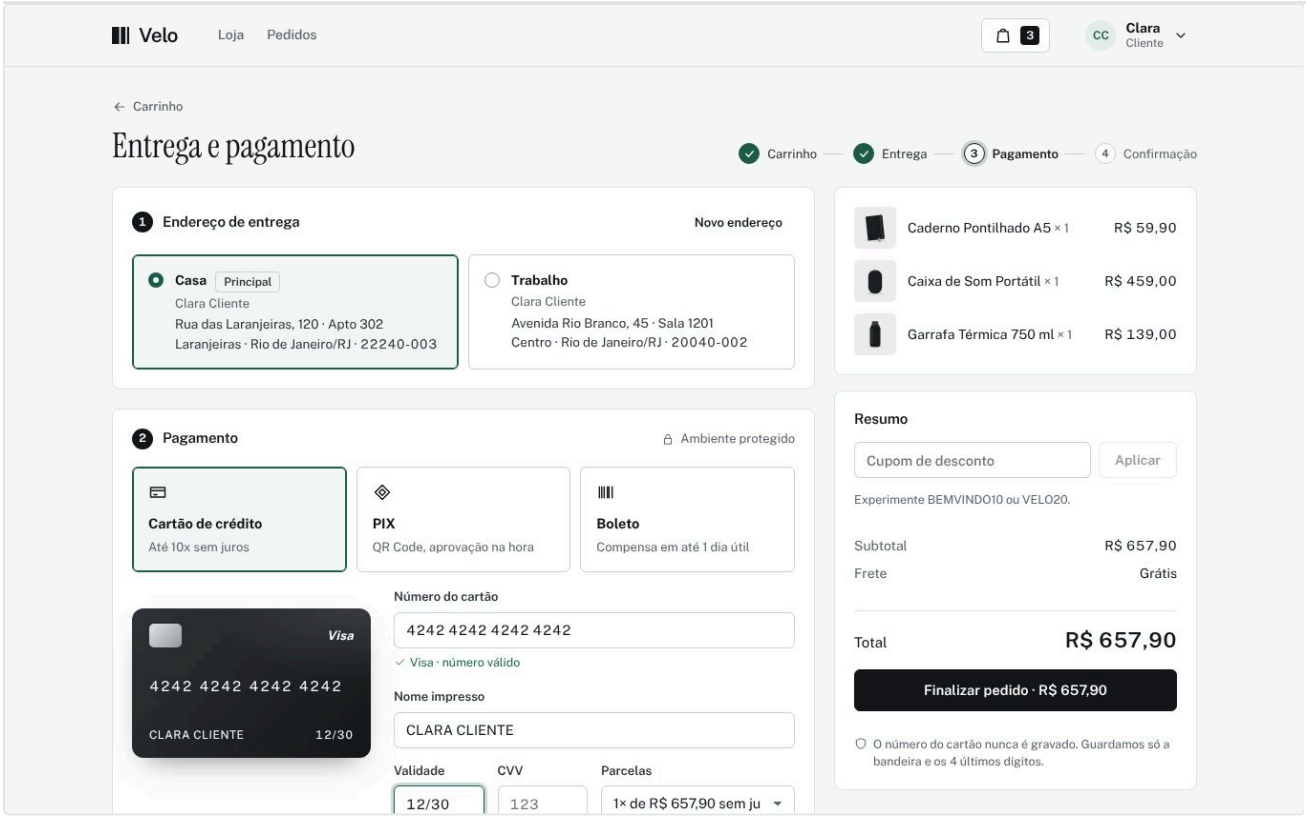
Página do carrinho com cupom e resumo

Entrega e pagamento

No topo do checkout, um indicador mostra em que etapa a compra está: **Carrinho**, **Entrega**, **Pagamento** e **Confirmação**. O cliente escolhe um endereço salvo ou cadastra um novo, escolhe a forma de pagamento e finaliza.

- **Endereço pelo CEP.** Ao digitar o CEP, rua, bairro, cidade e UF são preenchidos sozinhos. A consulta passa pela API da própria Velo, que pergunta ao ViaCEP; os campos continuam editáveis.
- **Prévia do cartão.** Um cartão desenhado acompanha o que é digitado e mostra a bandeira detectada pelo número.
- **Validação em tempo real.** O número é conferido pelo algoritmo de Luhn enquanto se digita; validade e CVV também são checados antes de enviar.
- **Prazo no resumo.** O resumo mostra subtotal, desconto, frete, total e o prazo estimado de entrega.

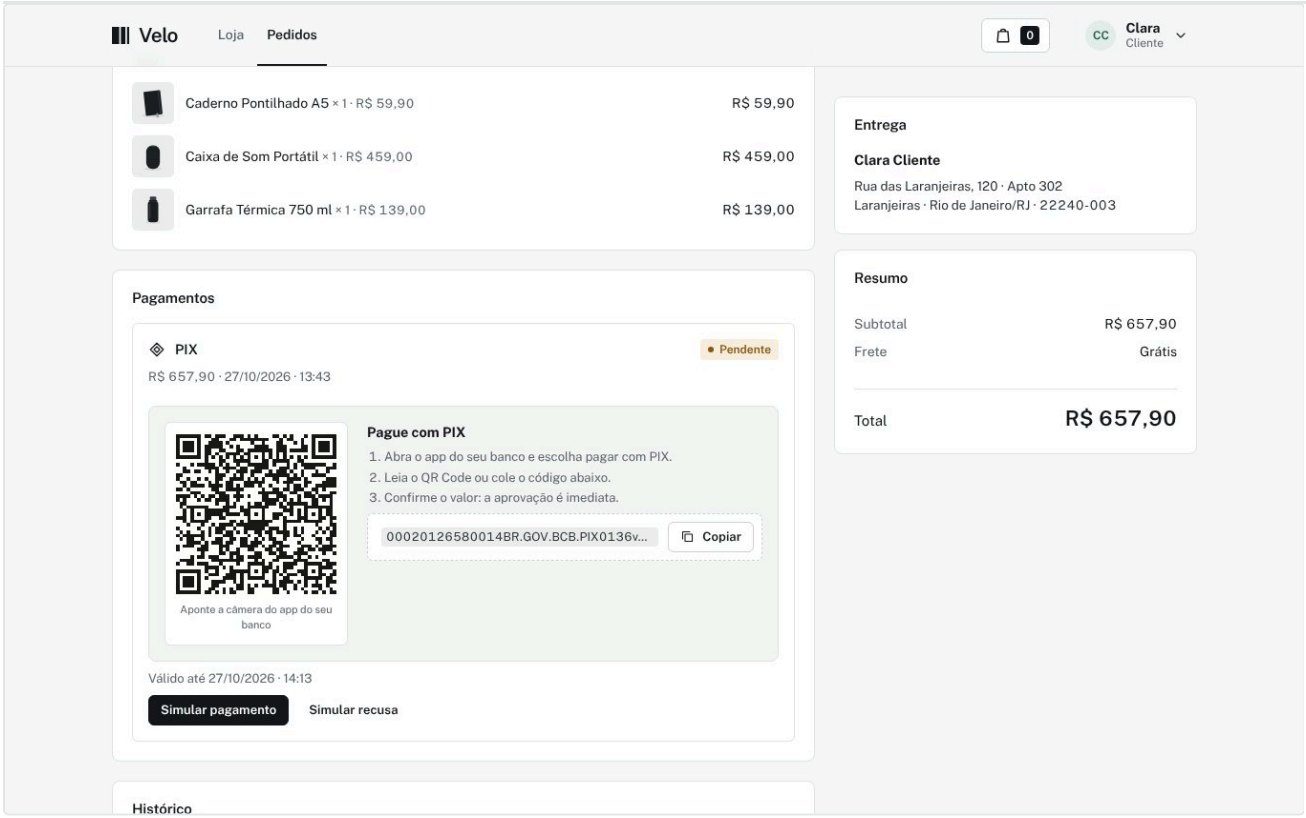
No modo simulado, o próprio formulário mostra os cartões de teste.



Checkout com indicador de etapas, endereço, prévia do cartão e resumo

Pagamento por PIX

Ao escolher PIX, o pedido mostra o QR Code para ler no aplicativo do banco e, logo abaixo, o código copia e cola com um botão para copiar. Um contador mostra quanto tempo falta para o código vencer (30 minutos).

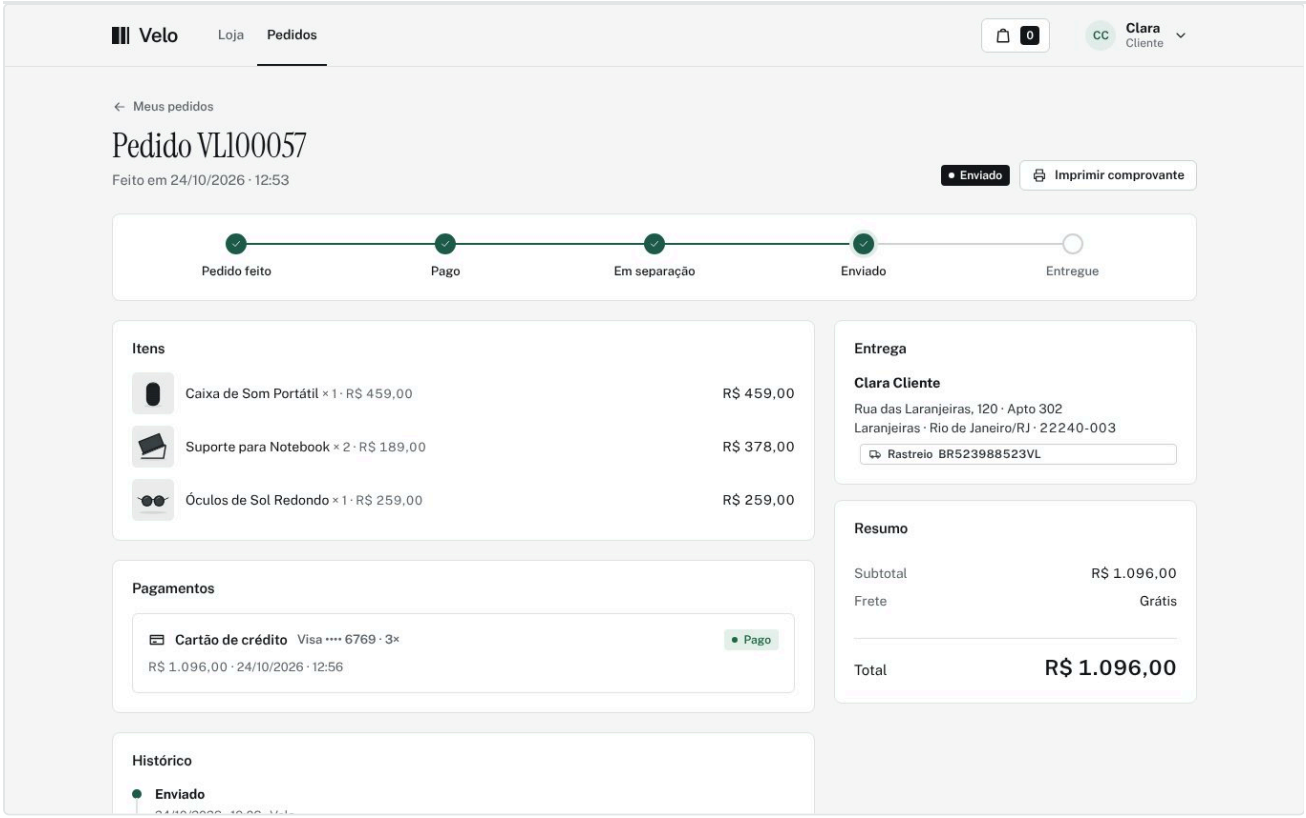


Pedido aguardando pagamento por PIX, com QR Code e copia e cola

Pedidos

A lista mostra todos os pedidos com filtro por situação. O detalhe traz a linha do tempo, os itens, os pagamentos, o endereço e o resumo. Enquanto o pedido aguarda pagamento, um contador mostra quanto falta para o cancelamento automático, e o botão **Pagar agora** permite tentar de novo.

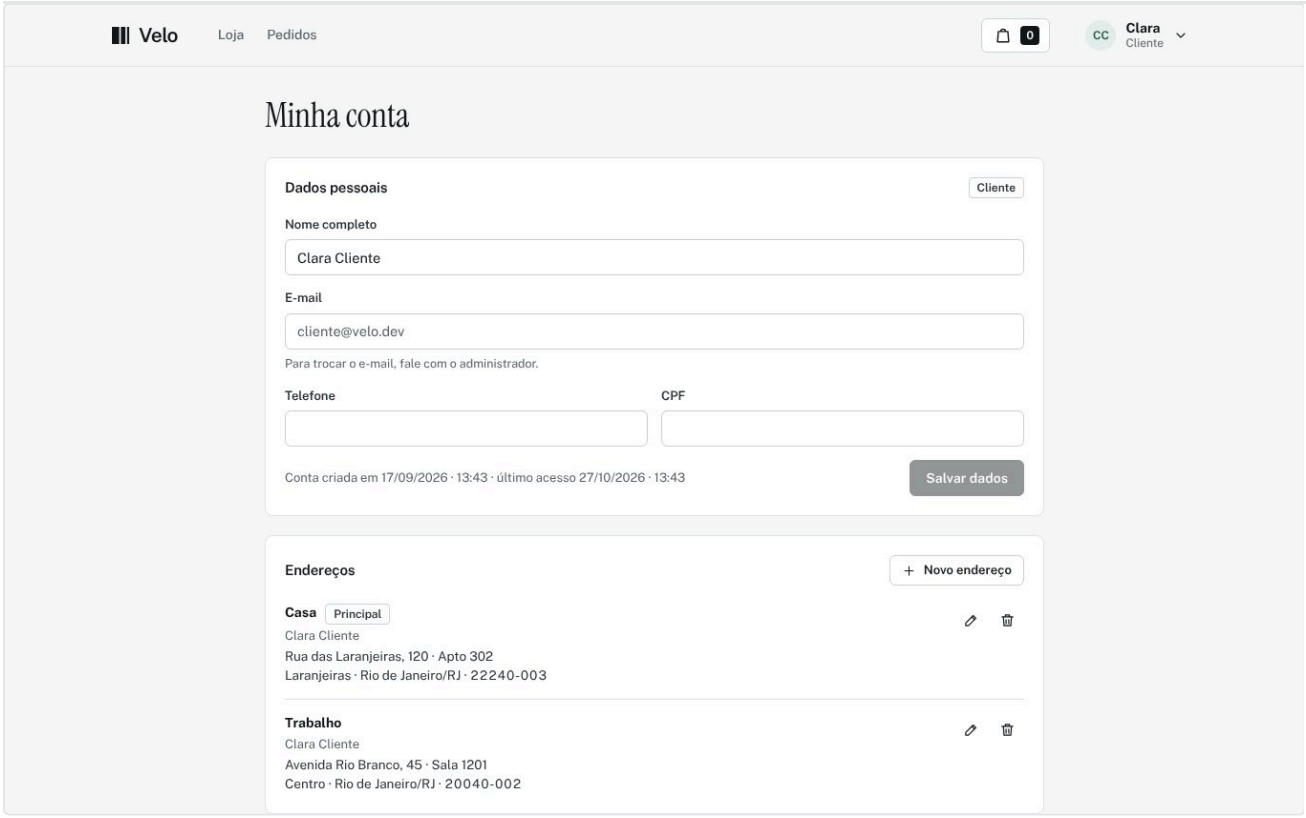
O botão **Imprimir comprovante** gera uma versão limpa do pedido, sem menus nem botões, pronta para imprimir ou salvar em PDF pelo próprio navegador (escolha **Salvar como PDF** na janela de impressão).



Pedido pago com linha do tempo e comprovante para impressão

Minha conta

Dados pessoais, endereços (com um principal) e troca de senha. O menu do usuário, no canto superior, mostra quanto tempo resta de sessão e traz a troca entre tema claro e escuro.



Minha conta com dados pessoais e endereços

No celular

Todas as telas se reorganizam em telas pequenas: a grade vira duas colunas, os formulários ficam em uma coluna e o painel passa a ter a navegação no topo.



Vitrine no celular

CAPÍTULO 5

Painel administrativo

O painel abre para os perfis Equipe e Administrador. A navegação fica numa barra lateral, e cada tela guarda os filtros no endereço, então dá para compartilhar um link já filtrado.

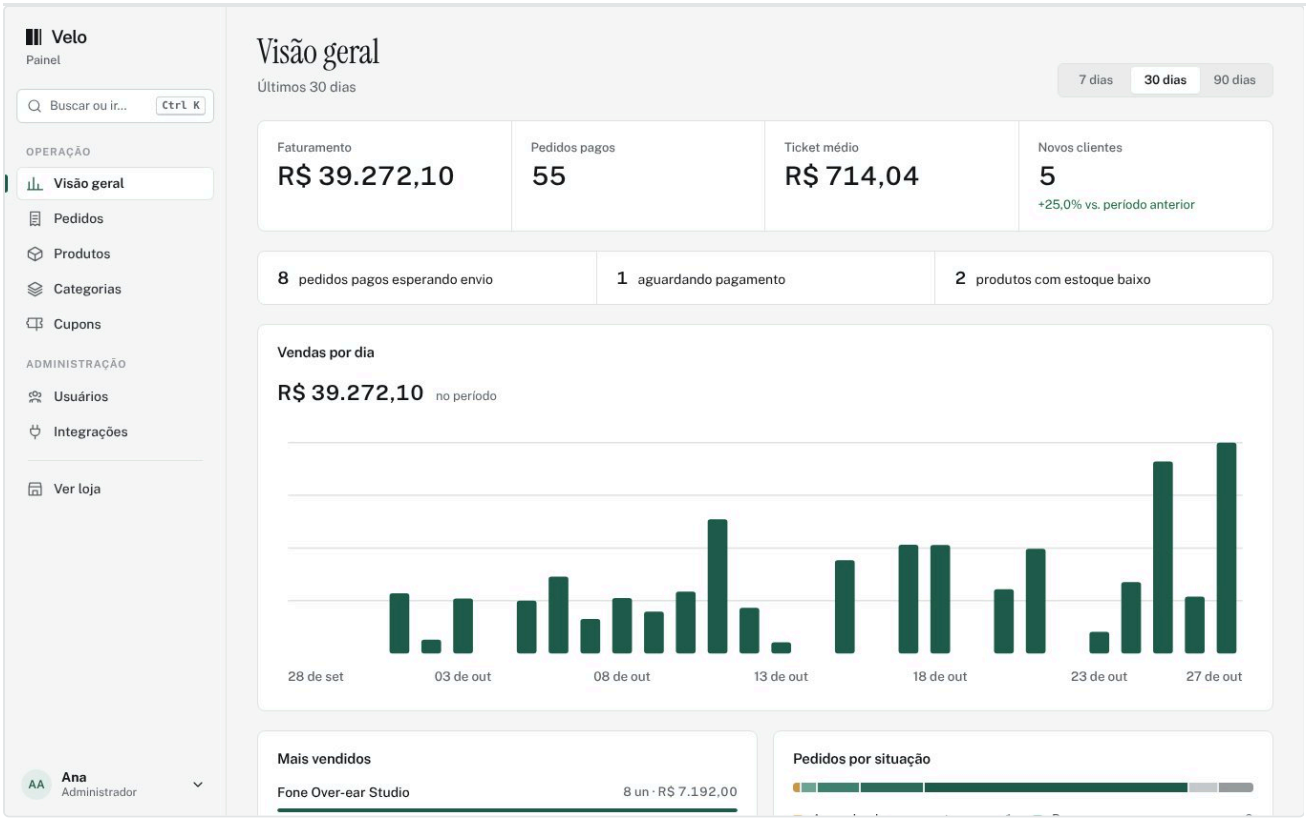
Perfis e permissões

Área	Cliente	Equipe	Administrador
Loja, carrinho, checkout, pedidos e conta	sim	sim	sim
Visão geral do painel e paleta de comandos	não	sim	sim
Pedidos: ver, mudar situação, cancelar, estornar e exportar	não	sim	sim
Produtos, estoque, categorias e exportação de produtos	não	sim	sim
Cupons	não	sim	sim
Usuários: criar, editar, bloquear, redefinir senha e encerrar sessões	não	não	sim
Integrações: provedores, webhooks e evento de teste	não	não	sim

Visão geral

Faturamento, pedidos pagos, ticket médio e novos clientes ficam numa faixa única de indicadores, com troca entre 7, 30 e 90 dias. Cada indicador mostra a variação contra o período anterior de mesmo tamanho, em verde quando sobe e em vermelho quando cai. Ao lado, o mix de formas de pagamento mostra quanto do faturamento veio de cartão, PIX e boleto.

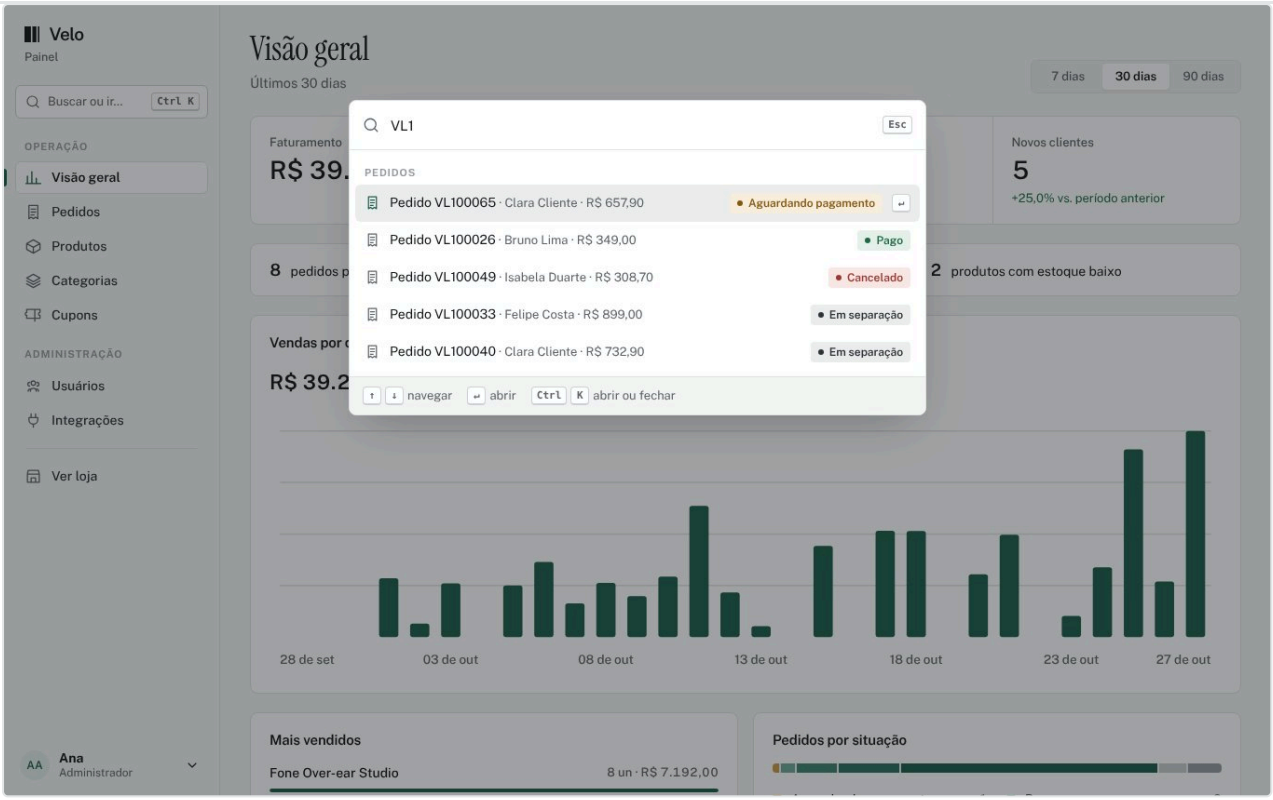
Logo abaixo ficam os atalhos para o que precisa de atenção, o gráfico de vendas por dia em barras verde-pinho (passe o mouse nas barras para ver o valor), os mais vendidos, os pedidos por situação, os pedidos recentes e o estoque baixo. Os avisos de cada ação aparecem no canto inferior direito, e quando a sessão está perto do limite uma faixa âmbar aparece no topo.



Visão geral do painel com variação por indicador e mix de pagamentos

Paleta de comandos NOVO

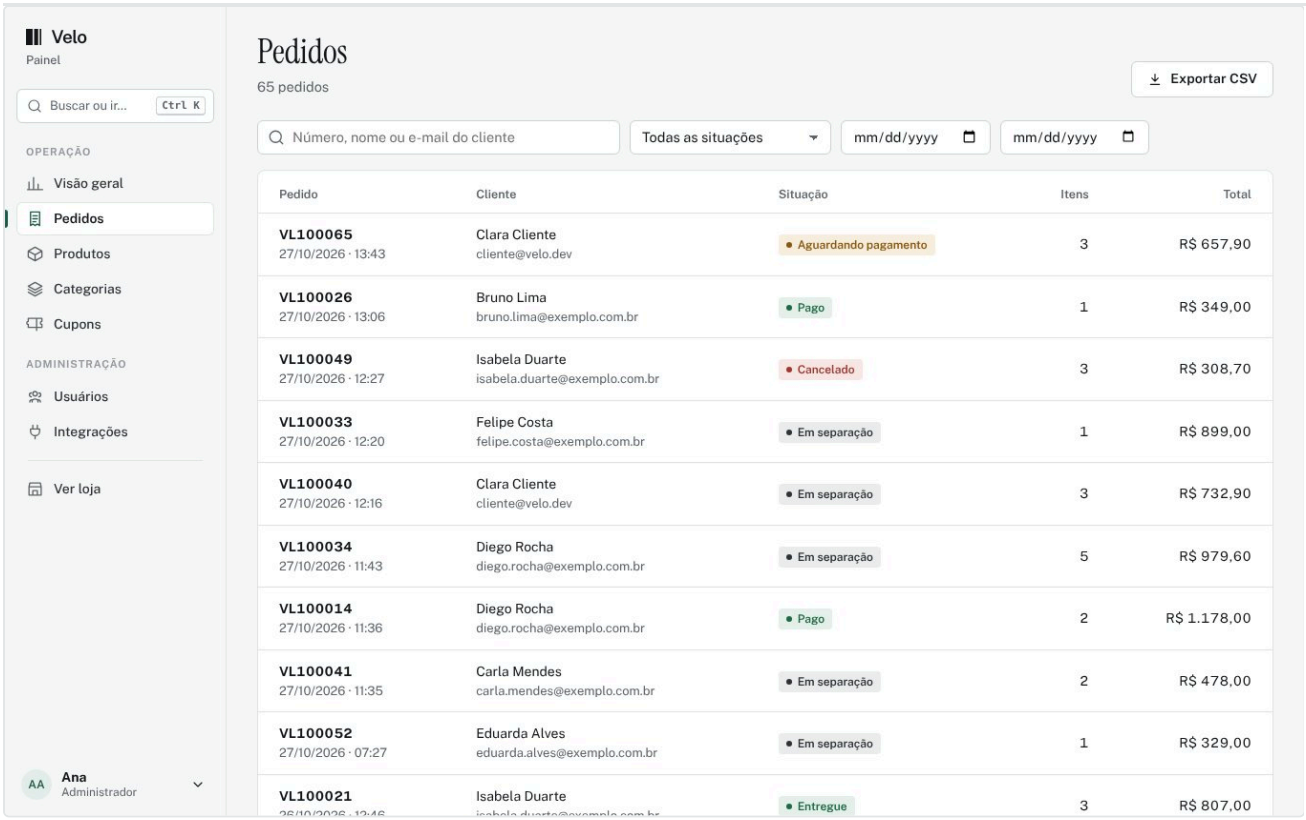
Em qualquer tela do painel, `Ctrl+K` (ou `⌘K` no Mac) abre a paleta de comandos. Digite o nome de uma tela para ir até ela, ou o número de um pedido (por exemplo, `VL104233`) para abri-lo direto. As setas escolhem, `Enter` confirma e `Esc` fecha.



Paleta de comandos aberta sobre o painel

Pedidos

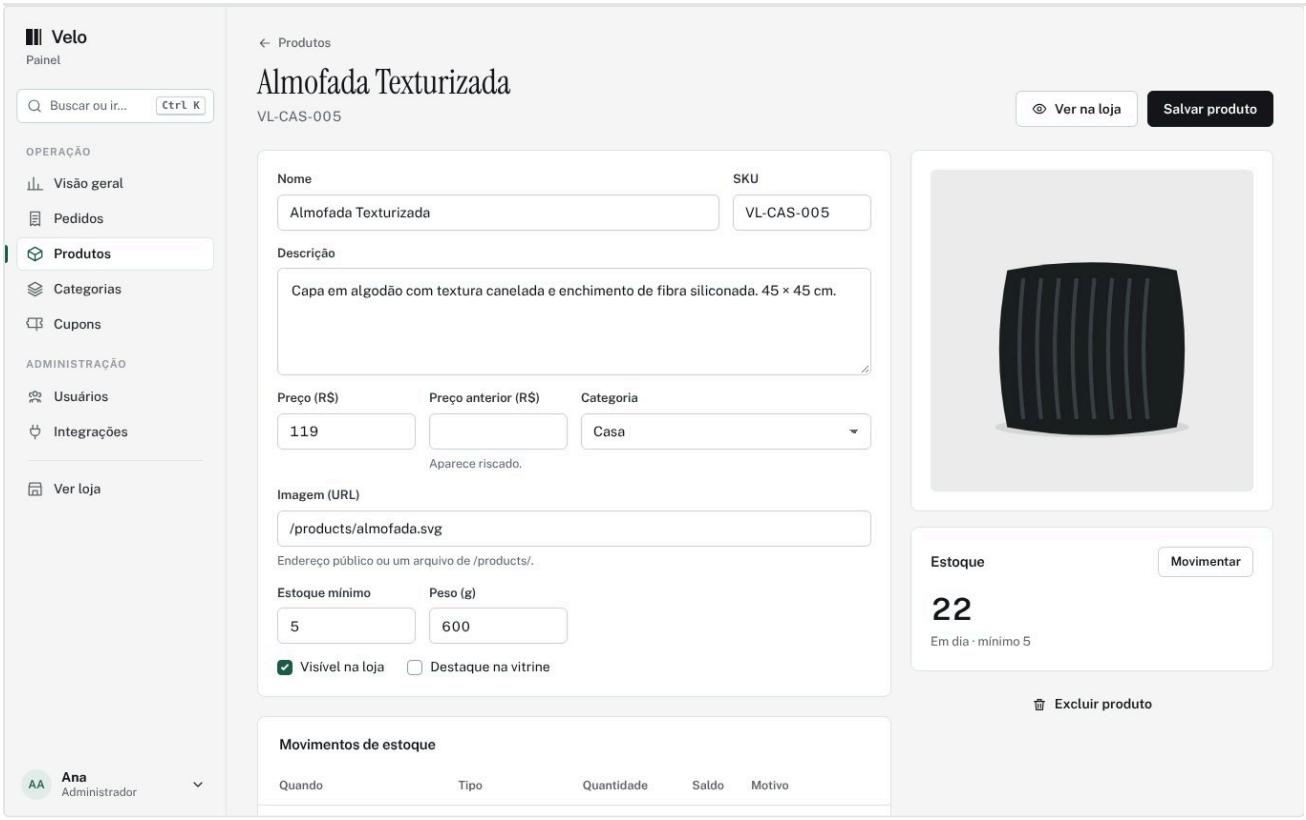
Busca por número, nome ou e-mail do cliente, filtro por situação e por período. No detalhe, os botões no topo mostram só as mudanças permitidas a partir da situação atual. Para marcar como enviado, o código de rastreio é obrigatório; cancelar ou estornar devolve o estoque e, se houve pagamento, o valor.



Lista de pedidos com filtros e exportação CSV

Produtos e estoque

A lista ordena por nome, menor estoque, mais recentes ou maior preço. No cadastro ficam nome, SKU (gerado sozinho se ficar vazio), descrição, preços, categoria, imagem, estoque mínimo, peso, visibilidade e destaque. O estoque só muda por movimentos: **Entrada**, **Saída** ou **Contagem**, e cada um fica registrado com o saldo e o motivo. Um produto que já aparece em pedidos não é apagado, só desativado, para manter o histórico.



Cadastro do produto com estoque e movimentos

Exportação CSV NOVO

As listas de pedidos e de produtos têm o botão **Exportar CSV**. O arquivo foi pensado para abrir direto no Excel com dois cliques, sem passar pelo assistente de importação.

Detalhe	Como é
Codificação	UTF-8 com BOM, para os acentos aparecerem certos no Excel
Separador	Ponto e vírgula (`;`)
Números	Vírgula decimal, sem separador de milhar (`1249,90`)
Pedidos	Respeita os filtros aplicados na tela: busca, situação e período
Produtos	Todos os produtos cadastrados, com SKU, categoria, preços e estoque
Rotas	`GET /api/admin/orders/export` e `GET /api/admin/products/export`

Categorias, cupons e usuários

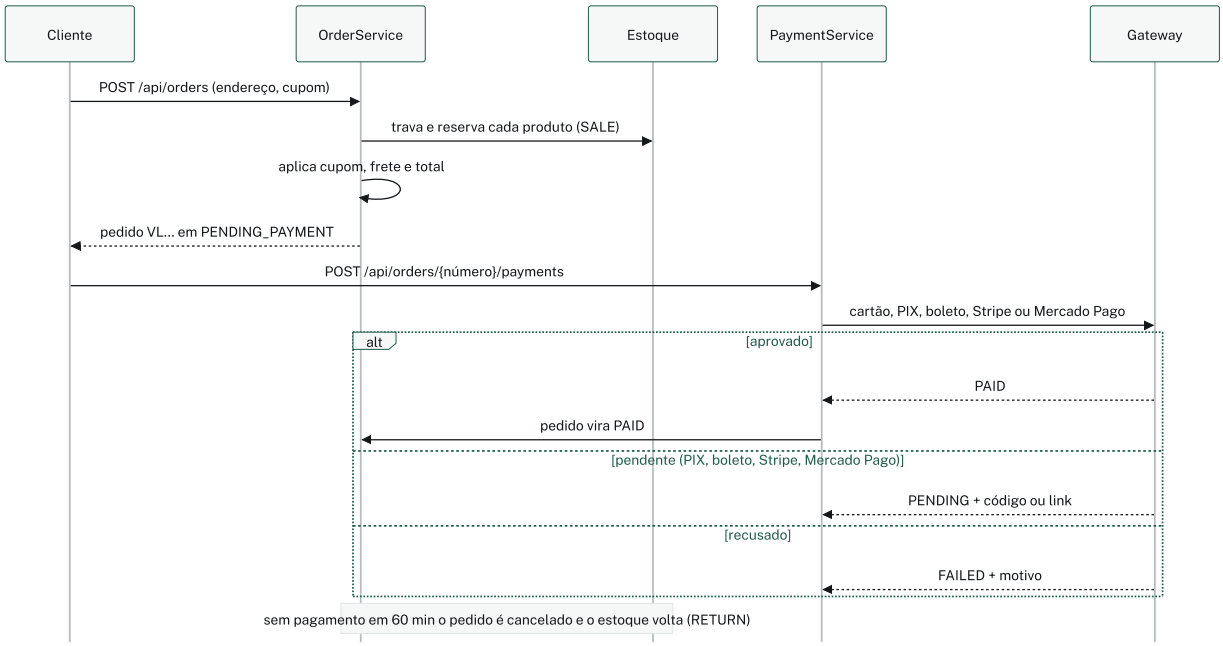
Categorias têm nome, descrição, ordem de exibição e visibilidade. Cupons podem ser percentuais ou de valor fixo, com pedido mínimo, limite de usos e período de validade. Na tela de usuários, o administrador cria contas, muda perfis, bloqueia o acesso, redefine senhas e encerra sessões abertas. A Velo impede que o último administrador ativo seja rebaixado ou bloqueado, e ninguém consegue tirar o próprio acesso de administrador.

CAPÍTULO 6

Pedidos e pagamentos

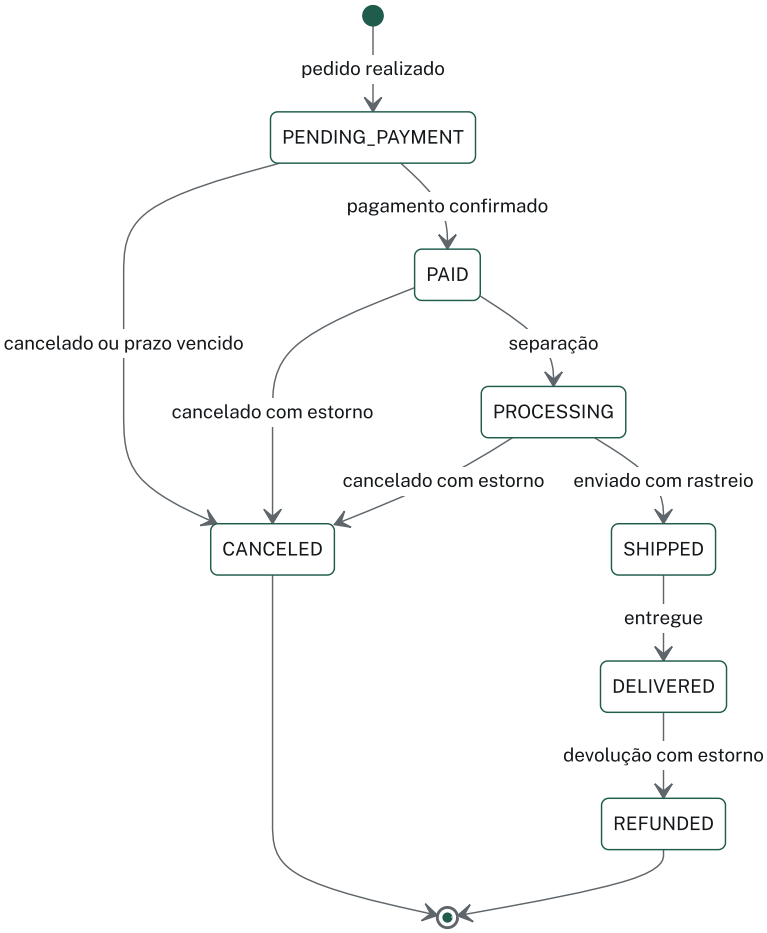
Como um pedido nasce, é pago e termina, e como cada forma de pagamento funciona.

Ciclo do pedido



Da finalização ao pagamento

Situação	Significado	Próximos passos possíveis
`PENDING_PAYMENT`	Aguardando pagamento	`PAID`, `CANCELED`
`PAID`	Pago	`PROCESSING`, `CANCELED` (com estorno)
`PROCESSING`	Em separação	`SHIPPED` (exige rastreio), `CANCELED`
`SHIPPED`	Enviado	`DELIVERED`
`DELIVERED`	Entregue	`REFUNDED` (devolução com estorno)
`CANCELED`	Cancelado	Fim
`REFUNDED`	Estornado	Fim



Fluxo de situações do pedido

Regras de valor e estoque

- O estoque de cada item é reservado com trava no banco na finalização (movimento `SALE`) e volta ao cancelar ou estornar (`RETURN`).
- Pedidos sem pagamento em 60 minutos são cancelados por uma rotina que roda a cada minuto.
- Frete fixo de R$ 19,90, grátis a partir de R$ 299 de subtotal. O prazo exibido é uma estimativa pela região do CEP (veja o capítulo Integrações).
- O cupom é validado de novo na finalização: precisa estar ativo, dentro do período, abaixo do limite de usos e acima do pedido mínimo.
- O pedido guarda uma cópia do endereço e do nome e preço de cada item; mudar um produto depois não altera pedidos antigos.
- Os números de pedido seguem o formato `VL` seguido de seis dígitos.
- Cada mudança de situação dispara o webhook de saída correspondente, se estiver configurado.

Pagamento simulado

É o modo padrão. Nenhum valor é cobrado. Use qualquer nome, validade futura e CVV de 3 dígitos:

Cartão	Bandeira	Resultado
`4242 4242 4242 4242`	Visa	Aprovado

Cartão	Bandeira	Resultado
`5555 5555 5555 4444`	Mastercard	Aprovado
`4000 0000 0000 0002`	Visa	Recusado pelo emissor
`4000 0000 0000 9995`	Visa	Saldo insuficiente

PIX gera o QR Code e o código copia e cola, válidos por 30 minutos; o boleto gera a linha digitável com vencimento em 3 dias. Os dois deixam o pedido aguardando, e o botão **Simular pagamento** no detalhe do pedido confirma como se o banco tivesse avisado. O número do cartão nunca é gravado: ficam só a bandeira e os 4 últimos dígitos.

Stripe

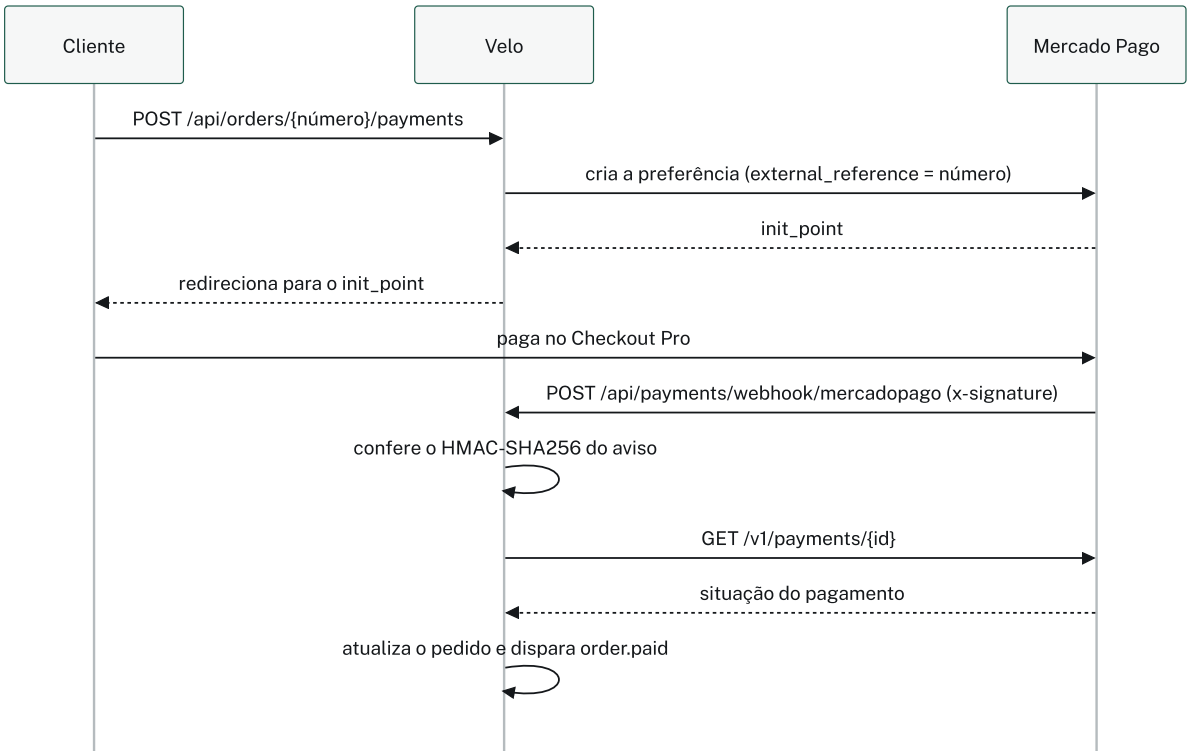
1. Crie uma conta em stripe.com e copie a chave secreta de teste (`sk_test_...`).
2. Em **Developers** › **Webhooks**, cadastre `https://SEU-ENDERECO/api/payments/webhook/stripe` com os eventos `checkout.session.completed` , `checkout.session.async_payment_succeeded` , `checkout.session.async_payment_failed` e `checkout.session.expired` . Copie o segredo de assinatura (`whsec_...`).
3. Defina as variáveis `VELO_PAYMENT_PROVIDER=STRIPE` , `STRIPE_SECRET_KEY` , `STRIPE_WEBHOOK_SECRET` e `VELO_PUBLIC_URL` e suba a aplicação.
4. Para testar localmente, use o Stripe CLI: `stripe listen --forward-to localhost:8080/api/payments/webhook/stripe` .

O checkout passa a levar o cliente para a página segura da Stripe. A confirmação chega pelo webhook, que tem a assinatura conferida (com tolerância de 5 minutos) antes de qualquer alteração no pedido. Ao voltar da Stripe, o cliente entra de novo, porque a página recarrega.

Mercado Pago NOVO

A Velo usa o **Checkout Pro** do Mercado Pago: o cliente paga numa página do próprio Mercado Pago, com cartão, PIX, boleto ou saldo da conta, e volta para a loja.

1. Crie uma conta em mercadopago.com.br e, em **Suas integrações**, crie uma aplicação. Copie o **Access Token** de teste (começa com `TEST-`) ou o de produção (`APP_USR-`).
2. Em **Suas integrações** › **(sua aplicação)** › **Webhooks**, cadastre `https://SEU-ENDERECO/api/payments/webhook/mercadopago` , marque o evento **Pagamentos** e salve. Copie a **assinatura secreta** mostrada na mesma tela.
3. Defina `VELO_PAYMENT_PROVIDER=MERCADO_PAGO` , `MERCADOPAGO_ACCESS_TOKEN` , `MERCADOPAGO_WEBHOOK_SECRET` e `VELO_PUBLIC_URL` e suba a aplicação.
4. Confira na tela **Integrações** do painel se o Mercado Pago aparece como ativo e copie de lá a URL do webhook, se ainda não tiver cadastrado.
5. Para testar localmente, exponha a porta 8080 com um túnel (por exemplo, `cloudflared` ou `ngrok`) e use esse endereço no webhook e em `VELO_PUBLIC_URL` (o Mercado Pago exige `https://`).



Pagamento com Mercado Pago Checkout Pro

Etapa	O que a Velo faz
Preferência	Cria a preferência com o total do pedido (item “Pedido VL… · Velo”) e `external_reference` igual ao número do pedido, e manda o cliente ao `init_point`. O retorno automático e a `notification_url` só são enviados quando `VELO_PUBLIC_URL` começa com `https://`.
Assinatura	Confere o cabeçalho `x-signature` (`ts=...,v1=...`): calcula o HMAC-SHA256 do manifesto `id:{data.id};request-id:{x-request-id};ts:{ts};` com a assinatura secreta. Como a situação é sempre consultada de novo, avisos repetidos não mudam nada; só os com mais de 48 horas são recusados.
Confirmação	Não confia só no aviso: consulta `GET /v1/payments/{id}` e só então atualiza o pedido.
Estorno	Cancelar ou estornar um pedido pago chama `POST /v1/payments/{id}/refunds`. Estornos e contestações feitos no painel do Mercado Pago também chegam pelo webhook e deixam o pedido como Estornado.
Prazo	A preferência vence junto com o prazo de pagamento do pedido (60 minutos), para PIX e boleto não serem pagos depois do cancelamento.
Pago depois do prazo	Se um pagamento aprovado chegar para um pedido já cancelado ou já pago, a Velo registra o valor e estorna na hora, sem deixar dinheiro cobrado sem pedido.

> **Nunca coloque chaves reais no código**
>
> Use variáveis de ambiente. O arquivo `.env.example` lista todas as que a Velo entende.

CAPÍTULO 7

Integrações

Como a Velo conversa com provedores de pagamento, com outros sistemas da loja e com o serviço de CEP, e como acompanhar tudo pelo painel.

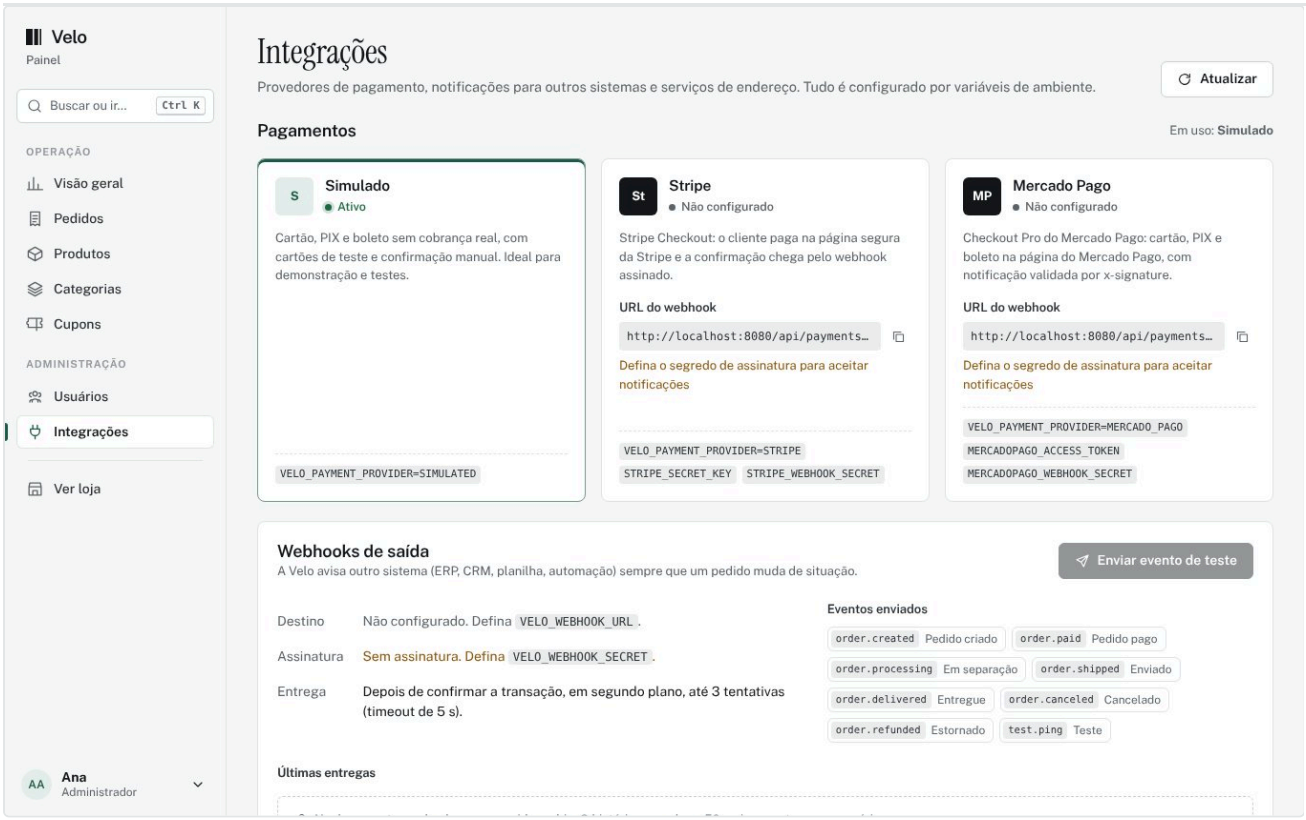
O que se integra

Integração	Para que serve	Como ligar
Stripe	Cobrança por Stripe Checkout	`VELO_PAYMENT_PROVIDER=STRIPE` + chaves `STRIPE_*`
Mercado Pago	Cobrança por Checkout Pro	`VELO_PAYMENT_PROVIDER=MERCADO_PAGO` + `MERCADOPAGO_*`
Webhooks de saída	Avisar ERP, CRM ou automações a cada mudança do pedido	`VELO_WEBHOOK_URL` e `VELO_WEBHOOK_SECRET`
ViaCEP	Preencher o endereço pelo CEP	Já vem ligado; não precisa de chave

Tela Integrações

Fica no painel e só abre para o perfil Administrador. Reúne num lugar o que antes só se via nas variáveis de ambiente:

- **Provedores de pagamento.** Situação do Simulado, da Stripe e do Mercado Pago, indicando qual está ativo e se as chaves foram informadas. Os segredos nunca aparecem por inteiro.
- **URLs de webhook.** Os endereços de entrada da Stripe e do Mercado Pago, já montados com `VELO_PUBLIC_URL` e prontos para copiar.
- **Webhooks de saída.** URL de destino, se há assinatura configurada e a lista de eventos enviados.
- **Enviar evento de teste.** Dispara um `test.ping` para a URL configurada e mostra a resposta.
- **Últimas entregas.** Histórico das 50 entregas mais recentes, com evento, horário, código HTTP, tentativas e duração. Fica em memória: reiniciar a aplicação apaga o histórico.
- **Consulta de CEP.** Um campo para testar o ViaCEP e ver o endereço que a loja receberia.



Tela Integrações com provedores, webhooks e histórico de entregas

Webhooks de saída

Servem para avisar outro sistema sempre que um pedido muda. Para ligar, defina a URL que vai receber os avisos e um segredo para a assinatura:

```
VELO_WEBHOOK_URL=https://erp.exemplo.com.br/velo/eventos
VELO_WEBHOOK_SECRET=um-segredo-longo-e-aleatorio
```

Sem `VELO_WEBHOOK_URL`, nada é enviado. Depois de configurar, use **Enviar evento de teste** na tela Integrações para conferir a ligação.

Eventos

Evento	Quando é enviado
`order.created`	Pedido finalizado, aguardando pagamento
`order.paid`	Pagamento confirmado
`order.processing`	Pedido em separação
`order.shipped`	Pedido enviado, com código de rastreio
`order.delivered`	Pedido entregue
`order.canceled`	Pedido cancelado, inclusive pelo prazo de pagamento vencido
`order.refunded`	Pedido estornado

Evento	Quando é enviado
`test.ping`	Botão Enviar evento de teste da tela Integrações

Corpo

POST COM CONTENT-TYPE: APPLICATION/JSON

```
{
  "id": "evt_…",
  "type": "order.paid",
  "createdAt": "2026-10-27T14:32:08Z",
  "data": {
    "number": "VL104233",
    "status": "PAID",
    "total": 349.90,
    "trackingCode": null,
    …
  }
}
```

Cabeçalhos

Cabeçalho	Conteúdo
`X-Velo-Event`	Tipo do evento, igual a `type` no corpo
`X-Velo-Delivery`	Identificador da entrega, útil para cruzar com o histórico da tela Integrações
`X-Velo-Signature`	`t=<segundos>,v1=<assinatura>`, onde a assinatura é o HMAC-SHA256, em hexadecimal, de `t.corpo` (o horário, um ponto e o corpo exato) com `VELO_WEBHOOK_SECRET`

Entrega e novas tentativas

- Os eventos só saem depois que a transação do pedido é gravada (depois do commit), em segundo plano: a resposta para o cliente nunca espera o outro sistema.
- Cada tentativa tem 5 segundos de limite.
- Em erro de rede ou resposta 5xx, a Velo tenta até 3 vezes no total, esperando 1 s e depois 3 s. Respostas 2xx encerram a entrega; outras respostas 4xx não são repetidas.
- Como uma entrega pode chegar repetida, use o `id` do evento para ignorar o que já foi processado.

Conferindo a assinatura

Quem recebe deve recalcular a assinatura com o corpo exatamente como chegou, antes de converter o JSON, e comparar em tempo constante. Recusar horários com mais de 5 minutos de diferença evita reenvio de avisos antigos.

PYTHON

```python
import hashlib, hmac, time

def assinatura_valida(corpo: bytes, cabecalho: str, segredo: str, tolerancia: int = 300) -> bool:
    partes = dict(p.split("=", 1) for p in cabecalho.split(",") if "=" in p)
    t, v1 = partes.get("t", ""), partes.get("v1", "")
    if not t.isdigit() or abs(time.time() - int(t)) > tolerancia:
        return False
    esperado = hmac.new(segredo.encode(), t.encode() + b"." + corpo, hashlib.sha256).hexdigest()
    return hmac.compare_digest(esperado, v1)

# corpo = bytes exatos recebidos; cabecalho = request.headers["X-Velo-Signature"]
```

NODE.JS

```node
import crypto from "node:crypto";

export function assinaturaValida(corpo, cabecalho, segredo, tolerancia = 300) {
  const p = Object.fromEntries(cabecalho.split(",").map((x) => x.split("=")));
  if (!p.t || Math.abs(Date.now() / 1000 - Number(p.t)) > tolerancia) return false;
  const esperado = crypto.createHmac("sha256", segredo).update(`${p.t}.${corpo}`).digest("hex");
  return typeof p.v1 === "string" && p.v1.length === esperado.length &&
    crypto.timingSafeEqual(Buffer.from(esperado), Buffer.from(p.v1));
}

// Express: app.post("/velo", express.raw({ type: "application/json" }), (req, res) => ...)
// use req.body (Buffer, sem conversão) como corpo.
```

CEP e prazo de entrega

A interface nunca chama o ViaCEP direto: ela pergunta à API da Velo, que consulta o serviço e devolve o endereço já no formato da loja. Assim a regra fica num lugar só e o navegador não depende de outro domínio.

Rota	O que devolve
`GET /api/address/cep/{cep}`	Logradouro, bairro, cidade e UF do CEP (8 dígitos, com ou sem hífen)
`GET /api/shipping/quote?cep=&subtotal=`	Valor do frete e prazo estimado em dias úteis para a região do CEP

O valor do frete segue a regra de sempre: R$ 19,90, ou grátis com subtotal acima de R$ 299. O prazo é uma estimativa pela faixa do CEP:

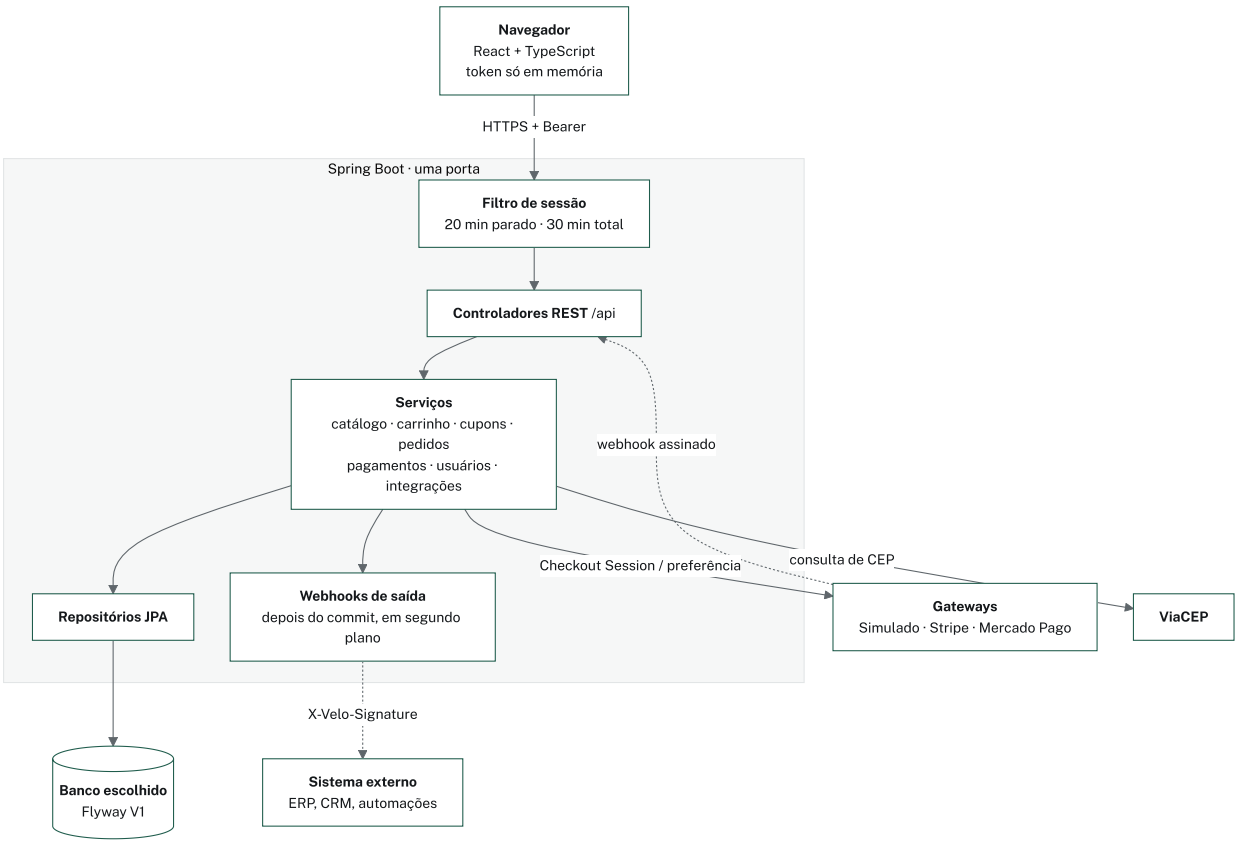
Sudeste	Sul	Centro-Oeste	Nordeste	Norte
2 a 4 dias úteis	3 a 5 dias úteis	4 a 7 dias úteis	5 a 9 dias úteis	7 a 12 dias úteis

Tocantins e Rondônia entram no Norte. Quando a faixa do CEP não identifica a UF, a resposta traz só a região.

CAPÍTULO 8

Arquitetura

A Velo é uma aplicação única: o Spring Boot serve a API em /api e a interface compilada na mesma porta.



Visão geral da arquitetura

Backend por pacote

Pacote	Responsabilidade
`auth`	Login, cadastro, sessões, filtro de autenticação e política de senha
`user`	Perfil, endereços e gestão de usuários pelo administrador
`catalog`	Categorias, produtos, busca, estoque, movimentos e exportação de produtos
`cart`	Carrinho de cada usuário
`coupon`	Cadastro, validação, uso e devolução de cupons
`order`	Checkout, pedidos, preços, ciclo de situações, cancelamento automático e exportação de pedidos
`payment`	Gateways simulado, Stripe e Mercado Pago, pagamentos, estornos e webhooks de entrada

Pacote	Responsabilidade
`integration`	Webhooks de saída (assinatura, novas tentativas, histórico) e a tela de Integrações
`shipping`	Consulta de CEP no ViaCEP com cache em memória e cotação de frete e prazo por região
`dashboard`	Indicadores, comparação com o período anterior e gráficos do painel
`seed`	Dados de demonstração na primeira execução
`config`	Segurança, propriedades, relógio e entrega da interface
`common`	Erros padronizados, paginação, dinheiro, textos e datas

Interface por pasta

Pasta	Conteúdo
`auth/`	Contexto de sessão, guardas de rota e avisos de tempo
`cart/`	Contexto do carrinho, compartilhado por todas as telas
`layouts/`	Estrutura da loja e do painel, gaveta do carrinho, menu do usuário e paleta de comandos
`pages/`	Entrada, vitrine, produto, carrinho, checkout, pedidos e conta
`pages/admin/`	Visão geral, pedidos, produtos, categorias, cupons, usuários e integrações
`features/`	Formulários de endereço e pagamento, prévia do cartão, blocos do pedido e gráficos
`components/`	Logo, ícones, modais, avisos e elementos básicos
`lib/`	Cliente da API, tipos, formatação, favoritos, vistos recentemente, tema e ganchos de dados

Decisões principais

- **Sessão opaca, não JWT.** O token é aleatório e o servidor guarda só o hash; assim dá para encerrar qualquer sessão na hora e aplicar os dois limites de tempo.
- **Token só em memória.** O token nunca vai para o armazenamento do navegador, então recarregar a página desloga, como pedido. Lá ficam só preferências da interface, como favoritos, vistos recentemente e tema.
- **Trava pessimista no estoque.** Dois clientes comprando a última unidade ao mesmo tempo não conseguem vender o mesmo item duas vezes.
- **Versão nas tabelas.** Produto, cupom e pedido têm coluna de versão para impedir que uma edição sobrescreva outra.
- **Webhooks depois do commit.** Os avisos de saída só partem quando a mudança já está gravada, em segundo plano, para nunca anunciar algo que foi desfeito nem atrasar o cliente.
- **Confirmação pela fonte.** Avisos de pagamento são assinados e, no Mercado Pago, ainda conferidos consultando o pagamento na API do provedor.
- **Esquema gerado.** Um script Python é a fonte única do modelo e gera o SQL de cada banco e a documentação.

- **Relógio injetável.** Toda regra de tempo usa um relógio do Spring, o que permite testar os limites de sessão sem esperar.

Erros

Toda resposta de erro tem o mesmo formato, e a interface mostra a mensagem e destaca os campos:

```
{ "code": "VALIDATION", "message": "Revise os campos destacados.",
  "fields": { "zipCode": "CEP inválido." } }
```

CAPÍTULO 9

API

Todas as rotas ficam em /api, respondem JSON e exigem o cabeçalho Authorization: Bearer com o token, exceto login, cadastro, saúde e os webhooks de pagamento.

Sessão e saúde

Método	Rota	Acesso	Para que serve
POST	`/api/auth/login`	público	Entrar e receber o token
POST	`/api/auth/register`	público	Criar conta de cliente
GET	`/api/auth/me`	logado	Dados de quem está logado
POST	`/api/auth/ping`	logado	Manter a sessão ativa
POST	`/api/auth/logout`	público	Encerrar a sessão
GET	`/api/health`	público	Saúde da aplicação

Loja e carrinho

Método	Rota	Acesso	Para que serve
GET	`/api/catalog/categories`	logado	Categorias visíveis
GET	`/api/catalog/products`	logado	Busca de produtos com filtros e paginação
GET	`/api/catalog/products/{slug}`	logado	Detalhe do produto
GET	`/api/cart`	logado	Carrinho com totais (aceita `?coupon=`)
POST	`/api/cart/items`	logado	Adicionar item
PUT	`/api/cart/items/{productId}`	logado	Mudar quantidade
DELETE	`/api/cart/items/{productId}`	logado	Remover item
DELETE	`/api/cart`	logado	Esvaziar o carrinho
GET	`/api/address/cep/{cep}` NOVO	logado	Endereço do CEP (via ViaCEP)
GET	`/api/shipping/quote?cep=&subtotal=` NOVO	logado	Valor do frete e prazo estimado

Pedidos e pagamentos

Método	Rota	Acesso	Para que serve
POST	`/api/orders`	logado	Finalizar a compra
GET	`/api/orders`	logado	Meus pedidos
GET	`/api/orders/{número}`	logado	Detalhe do meu pedido

Método	Rota	Acesso	Para que serve
POST	`/api/orders/{número}/payments`	logado	Pagar ou tentar de novo
POST	`/api/orders/{número}/cancel`	logado	Cancelar meu pedido
GET	`/api/payments/options`	logado	Formas de pagamento e cartões de teste
POST	`/api/payments/{id}/simulate`	logado	Confirmar ou recusar PIX/boleto simulado
POST	`/api/payments/webhook/stripe`	Stripe	Avisos de pagamento da Stripe (assinados)
POST	`/api/payments/webhook/mercadopago` NOVO	Mercado Pago	Avisos do Mercado Pago (públicos, assinados com `x-signature`)

Conta

Método	Rota	Acesso	Para que serve
GET, PUT	`/api/account`	logado	Ver e editar dados pessoais
PUT	`/api/account/password`	logado	Trocar a senha
GET, POST	`/api/account/addresses`	logado	Listar e criar endereços
PUT, DELETE	`/api/account/addresses/{id}`	logado	Editar e excluir endereço

Painel

Método	Rota	Acesso	Para que serve
GET	`/api/admin/dashboard`	equipe	Indicadores e comparação com o período anterior (aceita `?days=`)
GET	`/api/admin/orders`	equipe	Pedidos com busca e filtros
GET	`/api/admin/orders/export` NOVO	equipe	CSV dos pedidos, com os mesmos filtros da lista
GET	`/api/admin/orders/{número}`	equipe	Detalhe do pedido
PUT	`/api/admin/orders/{número}/status`	equipe	Mudar situação e rastreio
GET, POST	`/api/admin/products`	equipe	Listar e criar produtos
GET	`/api/admin/products/export` NOVO	equipe	CSV dos produtos
GET, PUT, DELETE	`/api/admin/products/{id}`	equipe	Ver, editar e excluir produto
POST	`/api/admin/products/{id}/stock`	equipe	Movimentar estoque
GET	`/api/admin/products/{id}/movements`	equipe	Histórico de estoque
GET, POST	`/api/admin/categories`	equipe	Listar e criar categorias
PUT, DELETE	`/api/admin/categories/{id}`	equipe	Editar e excluir categoria
GET, POST	`/api/admin/coupons`	equipe	Listar e criar cupons
PUT, DELETE	`/api/admin/coupons/{id}`	equipe	Editar e excluir cupom
GET, POST	`/api/admin/users`	admin	Listar e criar usuários
GET, PUT	`/api/admin/users/{id}`	admin	Ver e editar usuário

Método	Rota	Acesso	Para que serve
PUT	`/api/admin/users/{id}/password`	admin	Redefinir senha (encerra as sessões)
POST	`/api/admin/users/{id}/sessions/revoke`	admin	Encerrar sessões abertas
GET	`/api/admin/integrations` NOVO	admin	Provedores, URLs de webhook e últimas entregas
POST	`/api/admin/integrations/webhook/test` NOVO	admin	Enviar o evento `test.ping`

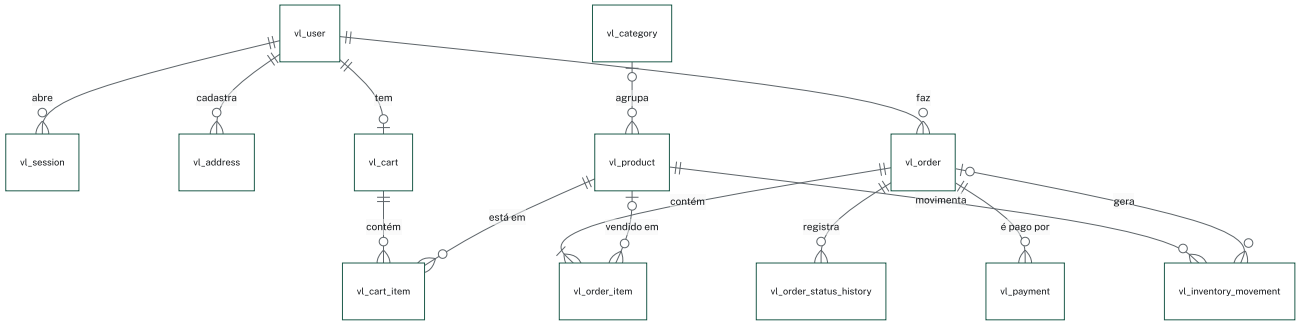
Respostas de sessão trazem os cabeçalhos `X-Velo-Idle-Expires-At` e `X-Velo-Expires-At`, usados pela interface para mostrar o tempo restante. As exportações respondem `text/csv` com o arquivo como anexo.

CAPÍTULO 10

Banco de dados

A Velo funciona com o banco que você preferir. As 13 tabelas são criadas sozinhas na primeira execução, com o Flyway.

O guia completo, com o diagrama de todos os campos e chaves, está no PDF **Banco de dados da Velo** (na raiz do projeto e em `docs/`) e em `docs/banco-de-dados.md`. Aqui fica o resumo. A versão 1.15.0 não muda o esquema: o histórico de webhooks fica em memória e os favoritos ficam no navegador.



Relacionamentos entre as tabelas

Pareando com outro banco

1. Suba o banco. Com Docker: `docker compose --profile postgresql up -d` (ou `mysql`, `mariadb`, `sqlserver`, `oracle`).
2. Sem Docker, crie o banco e o usuário `velo` com o SQL do guia.
3. Escolha o perfil: `SPRING_PROFILES_ACTIVE=postgresql` e, se o banco não estiver no endereço padrão, `VELO_DB_URL`, `VELO_DB_USER` e `VELO_DB_PASSWORD`.
4. Rode a aplicação. O Flyway cria as tabelas e a Velo cadastra os dados de demonstração.

Perfil	URL padrão	Usuário / senha
(nenhum)	`jdbc:h2:file:./data/velo`	`sa` / (vazia)
`postgresql`	`jdbc:postgresql://localhost:5432/velo`	`velo` / `velo`
`mysql`	`jdbc:mysql://localhost:3306/velo`	`velo` / `velo`
`mariadb`	`jdbc:mariadb://localhost:3307/velo`	`velo` / `velo`
`sqlserver`	`jdbc:sqlserver://localhost:1433;databaseName=velo`	`velo` / `Velo@Senha2026`
`oracle`	`jdbc:oracle:thin:@//localhost:1521/FREEPDB1`	`velo` / `velo`

CAPÍTULO 11

Configuração, testes e automação

Tudo o que muda de uma instalação para outra fica em variáveis de ambiente; o resto, em application.yml.

Variáveis de ambiente

Variável	Padrão	Para que serve
`SPRING_PROFILES_ACTIVE`	(vazio, H2)	`postgresql`, `mysql`, `mariadb`, `sqlserver` ou `oracle`
`VELO_DB_URL`	depende do perfil	Endereço JDBC do banco
`VELO_DB_USER` / `VELO_DB_PASSWORD`	depende do perfil	Credenciais do banco
`VELO_PORT`	`8080`	Porta do servidor
`VELO_SEED`	`true`	Cadastra os dados de demonstração quando o banco está vazio
`VELO_PUBLIC_URL`	`http://localhost:8080`	Endereço público, usado no retorno da Stripe e do Mercado Pago e nas URLs de webhook
`VELO_PAYMENT_PROVIDER`	`SIMULATED`	`SIMULATED`, `STRIPE` ou `MERCADO_PAGO`
`STRIPE_SECRET_KEY` / `STRIPE_WEBHOOK_SECRET`	(vazio)	Chaves da Stripe
`MERCADOPAGO_ACCESS_TOKEN` NOVO	(vazio)	Access Token da aplicação no Mercado Pago
`MERCADOPAGO_WEBHOOK_SECRET` NOVO	(vazio)	Assinatura secreta do webhook do Mercado Pago
`VELO_WEBHOOK_URL` NOVO	(vazio, desligado)	Destino dos webhooks de saída
`VELO_WEBHOOK_SECRET` NOVO	(vazio)	Segredo da assinatura `X-Velo-Signature`

Outras regras ficam em `application.yml`: limites de sessão, prazo de pagamento, frete, valor para frete grátis e número máximo de parcelas.

Testes

```
cd backend
./mvnw -B verify
```

Teste	O que confere
`SessionLimitsTest`	Encerramento por 20 minutos parado e por 30 minutos de vida, com relógio controlado

Teste	O que confere
`CheckoutFlowTest`	Compra com cartão aprovado e recusado, PIX, cupom, estoque, rastreio e cancelamento
`HttpApiTest`	Respostas 401 e 403, permissões de cada perfil, painel (com período anterior e mix de pagamentos), exportação CSV, tela de Integrações, CEP e cotação de frete
`UnitTests`	Validação de cartão (Luhn), textos, política de senha, assinaturas da Stripe e do Mercado Pago e faixas de CEP por região
`WebhookDeliveryTest`	Webhooks de saída contra um servidor HTTP local: assinatura, cabeçalhos, novas tentativas em 5xx e histórico de entregas

Integração contínua

A cada envio para a branch main, o GitHub Actions compila a interface e roda os testes do backend (`.github/workflows/ci.yml`).

Commits automáticos

O workflow `.github/workflows/diario.yml` registra uma linha em `.github/atividade.txt` e faz um commit todos os dias às 17h e às 23h (horário de Brasília), mantendo o gráfico de contribuições do GitHub ativo. Ele usa os segredos `GIT_NAME` e `GIT_EMAIL` do repositório; o passo a passo para ativar está no README.

Documentação em PDF

Este manual e o PDF do banco de dados são gerados por um script. Depois de mudar o conteúdo ou trocar as capturas em `docs/images/`, rode a partir da raiz do projeto:

```
python3 scripts/docs/gerar_pdfs.py
```

O script precisa de Python 3 com Playwright (Chromium) e pypdf, e grava os quatro arquivos: `Manual-Velo.pdf` e `Banco-de-Dados-Velo.pdf` na raiz e em `docs/`.

CAPÍTULO 12

Solução de problemas

Os sintomas mais comuns e o que fazer em cada caso.

Sintoma	O que fazer
`java` não é reconhecido	Instale o Java 21 e abra um novo terminal. Confira com `java -version`.
A porta 8080 já está em uso	Feche o outro programa ou defina `VELO_PORT=8081`.
Fui deslogado sozinho	É o esperado ao recarregar a página, depois de 20 minutos parado ou 30 minutos de sessão. Entre de novo.
E-mail bloqueado por alguns minutos	Houve 5 tentativas erradas. Espere 5 minutos ou peça ao administrador para redefinir a senha.
`Connection refused` ao iniciar	O banco escolhido não está rodando ou a porta é outra. Confira com `docker compose ps` ou ajuste `VELO_DB_URL`.
`Migrations have failed validation`	O banco já tem outro esquema. Use um banco vazio só para a Velo.
Quero começar do zero no H2	Pare a aplicação e apague a pasta `backend/data`.
A interface não aparece em localhost:8080	Rode sem `-DskipUi` para o Maven compilar e embutir a interface.
Pagamento Stripe não confirma	Confira o webhook, o segredo `whsec_` e se `VELO_PUBLIC_URL` aponta para um endereço acessível pela Stripe.
Pagamento Mercado Pago não confirma	Confira se o evento **Pagamentos** está marcado, se `MERCADOPAGO_WEBHOOK_SECRET` é a assinatura da mesma aplicação do Access Token e se `VELO_PUBLIC_URL` começa com `https://` (sem isso a `notification_url` não é enviada).
Webhook de saída não chega	Use **Enviar evento de teste** na tela Integrações e veja o código HTTP no histórico. Respostas 4xx não são repetidas; confira a URL e a autenticação do destino.
Assinatura `X-Velo-Signature` não confere	Calcule sobre o corpo bruto, antes de converter o JSON, e use o mesmo `VELO_WEBHOOK_SECRET`.
O endereço não preenche pelo CEP	O ViaCEP pode estar fora do ar ou sem acesso à internet a partir do servidor. Preencha os campos à mão; a compra segue normalmente.
CSV abre com acentos errados	Abra com dois cliques no Excel. Se importar por outro programa, escolha UTF-8 e o separador `;`.
Os favoritos sumiram	Eles ficam no navegador, por usuário. Outro navegador ou dados de navegação apagados começam uma lista nova.